A

Major Project

On

REAL-TIME AI POWERED DECISION SUPPORT SYSTEM FOR EMERGENCY MEDICAL SERVICES COORDINATION

(Submitted in partial fulfilment of the requirements for the award of Degree)

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

By

Gojji Ushasri	(227R1A1290)
Chinthalapani Raj Vignesh	(227R1A1277)
Rapala Pavan	(227R1A12C3)

Under the Guidance of

Mr. K. SRINU

Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Accredited by NAAC, NBA, Permanently Affiliated to JNTUH, Approved by AICTE, New Delhi) Recognized Under Section 2(f) & 12(B) of the UGCAct.1956, Kandlakoya (V), Medchal Road, Hyderabad-501401.

May, 2026.

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled "**REAL-TIME AI POWERED DECISION SUPPORT SYSTEM FOR EMERGENCY MEDICAL SERVICES COORDINATION**" being submitted by **G. Ushasri (227R1A1290), Ch. Raj Vignesh (227R1A1277) & R. Pavan (227R1A12C3)** in partial fulfilment of the requirements for the award of the degree of B.Tech in Information Technology to the Jawaharlal Nehru Technological University Hyderabad, during the year 2025-26.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

K. Srinu
Assistant Professor
INTERNAL GUIDE

HEAD OF THE DEPARTMENT

Dr. A. Raji Reddy
DIRECTOR

Signature of External Examiner

Submitted for viva voice Examination held on _____

ACKNOWLEDGEMENT

We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project, we take this opportunity to express our profound gratitude and deep regard to our guide **Mr. K. Srinu,** Assistant Professor for his exemplary guidance, monitoring and constant encouragement throughout the project work. The blessing, help and guidance given by him shall carry us a long way in the journey of life on which we are about to embark.

We take this opportunity to extend our heartfelt appreciation to the Project Review Committee (PRC) Coordinators— **Dr. J. Narasimharao, Mrs. K. Supriya Suhasini, Mr. D. Krishna Kumar, Mr. G. Shankar, Mrs. M. Siva Jyothi and Ms. G. Menaka** —for their unwavering support, insightful guidance, and valuable inputs, which played a crucial role in steering this project through its various stages.

Our sincere appreciation also goes to our Head of the Department, for continuous support and encouragement in ensuring the successful completion of our project.

We are deeply grateful to **Dr. A. Raji Reddy**, Director, for his cooperation throughout the course of this project. Additionally, we extend our profound gratitude to Sri. **Ch. Gopal Reddy**, Chairman, Smt. **C. Vasantha Latha**, Secretary and Sri. **C. Abhinav Reddy**, Vice-Chairman, for fostering an excellent infrastructure and a conducive learning environment that greatly contributed to our progress.

We also acknowledge and appreciate the guidance and assistance provided by the faculty and staff of **CMR Technical Campus**, whose contributions have been invaluable in bringing this project to fruition.

Lastly, we sincerely thank our families for their unwavering support and encouragement. We also extend our gratitude to the teaching and non-teaching staff of CMR Technical Campus for their guidance and assistance. Their contributions, along with the support of everyone who helped directly or indirectly, have been invaluable in the successful completion of this project.

G.Ushasri (227R1A1290)
Ch.Raj Vignesh (227R1A1277)
R.Pavan (227R1A12C3)

VISION AND MISSION

INSTITUTE VISION:

To Impart quality education in serene atmosphere thus strive for excellence in Technology and Research.

INSTITUTE MISSION:

1. To create state of art facilities for effective Teaching- Learning Process.

2. Pursue and Disseminate Knowledge based research to meet the needs of Industry & Society.

3. Infuse Professional, Ethical and Societal values among Learning Community.

DEPARTMENT VISION:

To produce technically proficient and skilled graduates in Information Technology and to prepare them to meet the needs of industry and society in a holistic learning environment.

DEPARTMENT MISSION:

1. To prepare students with the essential knowledge and contemporary abilities for solving emerging technological problems.

2. To train the students on cutting – edge computing facilities with industry interaction to enhance in practical exposure to real world situations.

3. To enhance the leadership qualities and team spirit with social values to serve the society.

ABSTRACT

Emergency Medical Services (EMS) play a crucial role in providing timely medical assistance during emergencies. However, delays in decision-making, inefficient resource allocation, and lack of real-time coordination can significantly impact patient outcomes. This project presents a Real-Time AI Powered Decision Support System for Emergency Medical Services Coordination, designed to enhance emergency response efficiency using artificial intelligence and data analytics.The proposed system integrates machine learning algorithms to analyze real-time emergency data, predict incident severity, and recommend optimal ambulance dispatch and hospital allocation. The system collects data from various sources such as emergency calls, patient information, and location data. After preprocessing the data, AI models are trained to classify emergencies and suggest appropriate medical responses.A web-based interface built using Flask enables users to upload datasets, train models, and obtain real-time predictions through an API. The system also provides visualization of performance metrics and graphs to help administrators evaluate model accuracy and system performance. By automating decision-making and improving coordination among emergency units, the proposed system aims to reduce response time, improve resource utilization, and ultimately enhance patient survival rates.This AI-driven approach demonstrates the potential of intelligent systems in transforming traditional EMS operations into a more efficient, data-driven, and responsive emergency healthcare infrastructure.

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
Figure 3.1	Project Architecture of Real-Time AI Powered Decision Support System for Emergency Medical Services Coordination	14
Figure 3.3.1	Use Case Diagram of Real-Time AI Powered Decision Support System for Emergency Medical Services Coordination	16
Figure 3.3.2	Class Diagram of Real-Time AI Powered Decision Support System for Emergency Medical Services Coordination	17
Figure 3.3.3	Sequence Diagram of Real-Time AI Powered Decision Support System for Emergency Medical Services Coordination	18
Figure 3.3.4	Activity Diagram of Real-Time AI Powered Decision Support System for Emergency Medical Services Coordination	19
Figure 4.2	Screenshot of features of the Dataset	22
Figure 5.1	Illustration of proposed real-time decision support system GUI application	42
Figure 5.2	Upload Dataset	43
Figure 5.3	Count plot of (a) diabetes status (b) hypertension	44

Figure 5.4	Confusion matrices obtained with hypertension data using Proposed Tao-tree model	44
Figure 5.5	Start server for decision support system admin panel	45

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
Table 6.2.1	Uploading Dataset	47
Table 6.2.2	Classification	47

TABLE OF CONTENTS

ABSTRACT **i**
LIST OF FIGURES **ii**
LIST OF TABLES **iv**
1. INTRODUCTION **1**
1.1 PROJECT PURPOSE 2
1.2 PROJECT FEATURES 2
2. LITERATURE SURVEY **4**
2.1 REVIEW OF RELATED WORK 8
2.2 DEFINITION OF PROBLEM STATEMENT 9
2.3 EXISTING SYSTEM 9
2.4 PROPOSED SYSTEM 10
2.5 OBJECTIVES 12
2.6 HARDWARE & SOFTWARE REQUIREMENTS 12
2.6.1 HARDWARE REQUIREMENTS 12
2.6.2 SOFTWARE REQUIREMENTS 13
3. SYSTEM ARCHITECTURE & DESIGN **14**
3.1 PROJECT ARCHITECTURE 14
3.2 DESCRIPTION 15
3.3 UML DIAGRAMS 16
3.3.1 USECASE DIAGRAM 16
3.3.2 CLASS DIAGRAM 17
3.3.3 SEQUENCE DIAGRAM 18
3.3.4 ACTIVITY DIAGRAM 19
4. IMPLEMENTATION **20**
4.1 ALGORITHMS USED 20
4.2 SAMPLE CODE 23
5. RESULTS & DISCUSSION **42**
6. VALIDATION **46**
6.1 INTRODUCTION 46
6.2 TEST CASES 47
6.2.1 UPLOADING DATASET 47
6.2.2 CLASSIFICATION 47
7. CONCLUSION & FUTURE ASPECTS **48**
7.1 PROJECT CONCLUSION 48
7.2 FUTURE ASPECTS 48
8. BIBLIOGRAPHY **50**
8.1 REFERENCES 50
8.2 GITHUB LINK 52

1.INTRODUCTION

1. INTRODUCTION

Emergency Medical Services (EMS) are responsible for providing immediate medical assistance to people during critical situations such as accidents, heart attacks, natural disasters, and other medical emergencies. The effectiveness of EMS largely depends on how quickly and efficiently emergency responders can reach the patient and provide the required medical support. However, traditional emergency response systems often face challenges such as delayed ambulance dispatch, lack of proper coordination between hospitals and emergency units, and inefficient use of available resources.

With the advancement of Artificial Intelligence (AI) and data analytics, intelligent systems can be developed to support faster and more accurate decision-making in emergency situations. AI technologies can analyze large volumes of historical and real-time data to identify patterns, predict emergency severity, and recommend appropriate actions. This capability helps emergency coordinators make better decisions regarding ambulance allocation, hospital selection, and patient prioritization.

The Real-Time AI Powered Decision Support System for Emergency Medical Services Coordination is designed to improve the efficiency and reliability of emergency medical responses. The system collects emergency-related data, processes it using machine learning techniques, and provides intelligent recommendations to assist EMS operators.

By analyzing factors such as patient condition, location, available ambulances, and hospital capacity, the system can suggest the most suitable response strategy.The proposed system also provides a web-based platform that allows users to upload datasets, preprocess data, train machine learning models, and generate predictions through an API-based service. In addition, the system visualizes model performance and emergency response insights using graphs and evaluation metrics.

By integrating AI-driven decision support with emergency medical services, the system aims to reduce response time, improve coordination between emergency units and hospitals, optimize resource utilization, and ultimately enhance patient survival rate.This project demonstrates how modern AI technologies can transform traditional emergency healthcare systems into smarter, faster, and more efficient service networks.

1.1 PROJECT PURPOSE

The purpose of the Real-Time AI Powered Decision Support System for Emergency Medical Services Coordination is to improve the efficiency, speed, and accuracy of emergency medical response through the use of artificial intelligence and data-driven decision making. In emergency situations, delays in ambulance dispatch, improper resource allocation, and lack of coordination between emergency units and hospitals can lead to serious consequences for patients. This project aims to develop an intelligent system that can analyze real-time emergency data and support decision-makers in selecting the best possible response strategy. The system uses machine learning techniques to process historical and real-time data, classify the severity of emergencies, and recommend appropriate medical actions such as ambulance dispatch and hospital routing.

Additionally, the project provides a web-based platform where users can upload datasets, preprocess data, train AI models, and generate predictions through a Flask-based API. The system also visualizes model performance using graphs and evaluation metrics to support analysis and improvement. By integrating AI technologies into emergency service operations, the project seeks to reduce response time, improve resource management, enhance coordination among emergency units, and ultimately increase the chances of saving lives

1.2 PROJECT FEATURES

User Authentication (Signup/Login):

The system provides a secure login and registration feature that allows authorized users to access the platform and manage emergency service data.

Dataset Upload:

Users can upload emergency medical datasets into the system. The uploaded data is used for analysis, training, and prediction of emergency situations.

Data Preprocessing:

The system performs preprocessing steps such as data cleaning, handling missing values, encoding categorical data, and normalizing features to prepare the dataset for machine learning models.

AI Model Training:

Machine learning algorithms are used to train models on the prepared dataset to analyze emergency conditions and support decision-making.

Real-Time Prediction via API:

The trained model is deployed using a Flask API, which allows real-time predictions based on new emergency data inputs.

Performance Evaluation:

The system evaluates the trained models using metrics such as accuracy, precision, recall, and F1-score to measure prediction performance.

Graphical Visualization:

The platform provides graphs and charts to visualize model performance, dataset insights, and prediction results.

Decision Support:

The system assists emergency service coordinators by providing intelligent recommendations for ambulance dispatch and hospital allocation.

User-Friendly Interface:

A simple web-based interface enables users to easily upload data, train models, and view results without requiring deep technical knowledge.

2.LITERATURE SURVEY

2. LITERATURE SURVEY

Almadani, et al. [6] Proposed an advanced technology, such as Artificial Intelligence (AI), Internet of Medical Things (IoMT), and blockchain, enhance decision-making processes in Emergency Medical Services (EMS) coordination. It highlights key innovations in AI-powered decision support systems (DSS), including real-time data analytics for optimized resource allocation and route planning in emergency response scenarios. The study examines the integration of hybrid technologies, such as AI-driven systems that combine data from ambulance GPS tracking, hospital capacity, and traffic conditions, to improve the efficiency and effectiveness of emergency services.

Aityan, et al. [7] Focused on early detection of life-threatening and time-sensitive diseases like sepsis, stroke, and heart attack, which are the major causes of death in emergency medicine. Additional training was conducted on a total of 600 cases (300 sepsis; 300 non-sepsis). The collective capability of the integrated LLMs is much stronger than each individual engine. Emergency cases can be predicted based on information from multiple sensors and streaming sources combining traditional IT (Information Technology) infrastructure with Internet of Things (IoT) schemes. Medical personnel compare and validate the AI models used in this work.

Neira-Rodado, et al. [8] Proposed a comprehensive and systematic approach to designing and optimizing EMS systems tailored for urban traffic accidents. By integrating Geographic Information Systems (GISs), hypercube queuing models, Economic Value Added (EVA) calculations, and multi-criteria decision-making (MCDM) techniques, we developed a model that balances service efficiency, financial sustainability, and equitable access to emergency care. The hypercube queuing model was applied to estimate key performance metrics, such as response time, coverage, and the GINI index for equity, under varying numbers of ambulances and demand scenarios.

Cvetkovic, et al. [9] Conducted using a questionnaire consisting of 7 sections and a total of 88 variables, distributed to and collected from 172 healthcare institutions (Public Health Centres and Hospitals). Statistical methods, including Pearson's correlation, multivariate regression analysis, and chi-square tests, were rigorously applied to analyse and interpret the data. Results: The results from the multivariate regression analysis revealed that the organization of working hours ($\beta = 0.035$) and shift work ($\beta = 0.042$) were significant predictors of EMS organization, explaining 1.9% of the variance ($R2 = 0.019$). Furthermore, shift work ($\beta = -0.045$) and working hours ($\beta = -0.037$) accounted for 2.0% of the variance in the number of EMS points

performed (R2 = 0.020). Also, the availability of ambulance vehicles (β = 0.075) and financial resources (β = 0.033) explained 4.1% of the variance in mass casualty preparedness (R2 = 0.041). When it comes to service area coverage, the regression results suggest that none of the predictors were statistically significant.

Strauss, et al. [10] Analysed to draws an almost ten years of improvement in EMS' key performance indicators such as response time or overtime in Switzerland, using Discrete Event Simulation. Three representative simulation studies are used to reflect on the optimization potential of alternative bases and rosters, methodological limitations, and the uptake of the derived recommendations. The results demonstrate that EMSs' efficiency gains in resource utilization increasingly come into conflict with emergency departments' and healthcare authorities' policies, indicating a need to enrich Discrete Event Simulation with a systemic perspective.

Liu, et al. [11] Compound disaster emergency collaboration network (ECN) was constructed by identifying the interactional relationships between emergency organizations. After applying time slices, the dynamic evolution of network structure, organizational–functional relations, organizational attributes, and cross-organizational relationships were discussed. The density and connectivity of the compound disaster ECN first decreased before increasing. Meanwhile, the evolution of the network structure followed a path from decentralized to concentrated and from being uneven to an equilibrium.

Robakowska, et al. [12] Aimed to analyse the feasibility of using UAVs to support emergency medical systems in the supply and urgent care ranges. The implementation of drones in the medical security system requires proper planning of service cooperation, division of the area into sectors, assessment of potential risks and opportunities, and legal framework for the application. A systematic literature search was conducted to assess the applicability based on published scientific papers on possible medical drone applications in the field of urgent mode. The widespread applications of UAVs in healthcare are concerned with logistics, scope, and transportability, with framework legal constraints to effectively exploit opportunities for improving population health, particularly for costly critical situations.

kuttan, et al. [13] Focused on its capacity to enhance diagnostic precision, improve triage systems, and tailor treatment strategies. Emergency departments worldwide are increasingly adopting AI-driven tools, including advanced triage systems, predictive analytics, and automated diagnostic support. These technologies have shown impressive abilities in medical image analysis, patient outcome prediction, and clinical documentation assistance. Nevertheless, the implementation of AI in emergency medicine faces obstacles such as data accessibility and quality, ethical issues, and the need for comprehensive regulatory frameworks.

Al-Araji, et al. [14] Proposed an AI-powered Clinical Decision Support System (CDSS) designed to assist clinicians in the early detection of stroke using structured patient data such as vital signs, medical history, and neurological observations. The system leverages machine learning techniques to classify patient cases as likely stroke or non-stroke, providing real-time diagnostic support during triage or initial evaluation. To evaluate the effectiveness of the proposed approach, we develop a simulation environment that mimics emergency department workflows and incorporates both synthetic and publicly available clinical datasets.

Kiran Kumar Jaghni, et al. [15] Proposed HealthNavAI, a regulation-backed, nationwide Healthcare Service Availability and Routing Platform that mandates real-time, standardized data sharing from all healthcare providers. The platform integrates ED queue status, specialist appointments, diagnostic imaging capacity, elective surgery schedules, and ambulance availability. An AI-based routing engine processes this data to predict service load, optimize patient allocation, and recommend optimal service locations based on predicted wait times, travel distance, clinical capability, and equity constraints.

Schmidt Batista, et al. [16] Examined the impact of AI medical assistants on clinical productivity, patient outcomes, and administrative operations. Through a systematic literature review of peer-reviewed studies, case analyses, and empirical evaluations, we identify core use cases where AI contributes measurable gains, such as enhanced documentation accuracy, optimized triage, and reduced clerical workloads. These systems, often integrated with Electronic Health Records (EHRs), enable real-time data capture, automated symptom screening, and tailored treatment suggestions. Despite their benefits, adoption is constrained by algorithmic bias, data governance challenges, and professional resistance.

Lujak, et al. [17] presented a multi-agent decision-support system for the distributed coordination of EMS focusing on urgent out-of-hospital STEMI patients awaiting angioplasty. The system is also applicable to emergency patients of any pathology needing pre-hospital acute medical care and urgent hospital treatment. The assignment of patients to ambulances and angioplasty-enabled hospitals with cardiology teams is performed via a three-level optimization model. At each level, we find a globally efficient solution by a modification of the distributed relaxation method for the assignment problem called the auction algorithm. The efficiency of the proposed model is demonstrated by simulation experiments.

Tang et al. [18] Multi-disciplinary Collaboration (MC) framework, show the disruptive potential of LLMs in carrying out specialized, professional-level activities within the healthcare sector. These new frameworks demonstrate the ability of LLMs to perform zero-shot reasoning and facilitate collaborative multi-agent discourse, both of which are critical in high-stakes

contexts like EMS, where rapid and accurate decision-making is required. Nonetheless, considerable obstacles exist in the area of call regulation.

Miller et al. [19] demonstrated the significance of data linking by combining many datasets (ambulance, emergency department, hospital, etc.) to monitor trauma patients from the prehospital incident until discharge. This strategy improves patient care by increasing the availability and quality of prehospital data in hospitals, as well as allowing for improved monitoring and assessment of trauma systems.

Elfahim, O et al [20] The emergency medical services (EMS) sector, as a complex system, presents substantial hurdles in providing excellent treatment while operating within limited resources, prompting greater adoption of artificial intelligence (AI) as a tool for improving operational efficiency. While AI models have proved beneficial in healthcare operations, there is limited explainability and interpretability, as well as a lack of data used in their application and technological advancement.

Zota, R.D et al. [21] The rising complexity and operational demands of modern healthcare systems have significantly increased resource usage and associated costs. This trend highlights the need for innovative approaches to optimize workflows and enhance decision-making. From this perspective, the present study explores how artificial intelligence (AI) can contribute to improving efficiency and information access in the medical field. The article begins with an introduction and a concise literature review focused on the integration of AI in healthcare platforms.

Nickel et al. [22] approached the problem by aiming to minimize expected costs, primarily focusing on the fixed costs associated with ambulance bases while ensuring a specified service level. Several studies have also approached EMS optimization as a multi-objective problem by integrating multiple objectives. These studies employed various methods to weigh different objective functions, such as the GINI coefficient, survival rates, uncovered demand, penalties, relocation costs, and system preparedness.

Neira-Rodado et al. [23] Emergency medical services (EMSs) are critical to reducing fatalities and improving patient outcomes in emergencies such as traffic accidents, where response time is a decisive factor. This study proposes a comprehensive and systematic approach to designing and optimizing EMS systems tailored for urban traffic accidents. By integrating Geographic Information Systems (GISs), hypercube queuing models, Economic Value Added

(EVA) calculations, and multi-criteria decision-making (MCDM) techniques, we developed a model that balances service efficiency, financial sustainability, and equitable access to emergency care.

2.1 REVIEW OF RELATED WORK

Emergency Medical Services (EMS) have been the focus of many research studies aiming to improve response time, resource management, and patient outcomes. Various technologies such as artificial intelligence, machine learning, and real-time data analysis have been applied to enhance emergency response systems.

Several researchers have explored the use of machine learning algorithms to predict emergency incidents and classify the severity of medical conditions. These models analyze historical emergency data to identify patterns and help emergency operators make faster and more accurate decisions. Techniques such as Naïve Bayes, Decision Trees, and Neural Networks have been widely used for emergency prediction and classification tasks.

1.Data Collection Approach

The system collects data from various sources such as emergency calls, patient details, ambulance location data, and hospital availability. This data is essential for analyzing emergency situations and making accurate predictions.

2. Data Preprocessing Approach

Before using the data for analysis, the system performs preprocessing steps such as cleaning the data, removing missing values, encoding categorical data, and normalizing features. This improves the quality and reliability of the dataset used for training the models.

3. Machine Learning Approach

Machine learning algorithms are applied to analyze the emergency data and predict the severity of incidents. These models learn patterns from historical data and help classify emergency cases and recommend appropriate responses.

4. Real-Time Prediction Approach

The trained machine learning models are integrated with a Flask-based API to provide real-time predictions. When new emergency data is entered into the system, the model quickly processes it and generates recommendations for emergency response.

5. Decision Support Approach

The system provides intelligent suggestions for ambulance dispatch, hospital selection, and emergency prioritization. This helps emergency coordinators make faster and more informed decisions.

6. Visualization and Monitoring Approach

The system includes graphical visualization tools that display model performance metrics, prediction results, and data insights. These visualizations help administrators monitor system performance and understand the results more clearly.

7. Web-Based System Approach

A user-friendly web interface allows users to interact with the system easily. Users can upload datasets, train models, run predictions, and view results through the web platform.

2.2 DEFINITION OF PROBLEM STATEMENT

Traditional EMS coordination often relies on human judgment, which can lead to delays, errors, or suboptimal use of resources, especially in time-critical situations. The problem arises from the dynamic nature of emergency responses, where multiple variables (such as traffic conditions, patient health status, ambulance availability, and hospital capacity) must be considered simultaneously. EMS personnel struggle to make real-time decisions based on fragmented information, leading to inefficiencies in patient routing, resource allocation, and hospital triage. The problem is compounded by human fatigue, alert overload, and system integration challenges. Real-time, AI-driven solutions offer the potential to improve EMS coordination, but their implementation presents technical, ethical, and operational hurdles that need to be addressed.

2.3 EXISTING SYSTEM

The traditional system follows a linear and reactive workflow, where incident data is manually collected, ambulance units are dispatched based on availability lists, and hospitals are contacted through telephonic confirmation.Cannot perform simultaneous multi-target predictions or handle imbalanced data. Relies on manual evaluation and rule-based protocols without cloud connectivity. In addition, traditional EMS decision-making relied on bedside clinical judgment, paper protocols, and static database systems. . The traditional system follows a linear and reactive workflow, where incident data is manually collected, ambulance units are dispatched based on availability lists, and hospitals are contacted through telephonic confirmation. There is minimal automation, no predictive analytics, and limited capability to handle multiple emergencies simultaneously.

Limitations of Existing System

- Traditional systems cannot integrate or process real-time data such as patient vitals, ambulance location, or hospital availability. As a result, decisions are based on outdated information, leading to delayed or inappropriate responses.
- Every critical decision from triage to dispatch relies on human operators. Fatigue, cognitive overload, or subjective bias can compromise the accuracy of decisions, particularly in high-pressure emergency scenarios.
- Ambulances may be dispatched inefficiently due to the absence of algorithmic optimization. This often results in uneven workload distribution, increased fuel costs, and delayed medical intervention.
- Traditional frameworks do not incorporate machine learning or predictive models. They cannot predict patient outcomes, disease progression, or emergency risk levels (e.g., hypertension spikes or diabetic shock), limiting proactive healthcare measures.
- The system handles fragmented, inconsistent, and manually entered data. There is no normalization, encoding, or automated cleaning, which leads to inaccurate reporting and unreliable trend analysis.
- Inter-departmental communication between dispatch centres, ambulances, and hospitals is slow and fragmented. Information is transmitted sequentially rather than through real-time, cloud-based synchronization.

2.4 PROPOSED SYSTEM

The study focuses on developing an intelligent, data-driven approach to support timely decision-making in emergency medical scenarios by analyzing patient health data using machine learning techniques. With the increasing burden of chronic conditions such as hypertension and diabetes, healthcare systems require faster and more reliable analytical tools that can assist medical professionals during critical situations. This research emphasizes the use of computational intelligence to extract meaningful patterns from medical datasets and transform them into actionable insights that can support rapid clinical responses. The proposed study integrates data preprocessing, feature encoding, and class imbalance handling to ensure the quality and reliability of the medical data used for analysis. Since real-world healthcare datasets often contain missing values, categorical attributes, and uneven class distributions, advanced preprocessing and clustering-based oversampling techniques are applied to improve learning effectiveness. These steps help in creating balanced and structured data representations that enhance the predictive capability of machine learning models.

Advantages of the Proposed System

- **Faster Emergency Decision-Making:**
  The system uses artificial intelligence to quickly analyze patient data and support faster decision-making during emergency situations.
- **Accurate Health Predictions:**
  By using advanced machine learning algorithms such as the TaoTree Classifier, the system can accurately predict Blood Pressure Category and Diabetes Status, which helps in better medical assessment.
- **Real-Time Analysis:**
  The system processes patient and emergency data in real time, enabling quick responses and improved coordination between emergency medical services.
- **Improved EMS Coordination:**
  It helps emergency coordinators efficiently manage ambulances, hospitals, and medical resources, reducing delays in patient care.
- **Enhanced Healthcare Response:**
  Early prediction of critical health conditions allows medical teams to prepare appropriate treatment before the patient reaches the hospital.
- **Secure Data Management:**
  The system focuses on secure handling of patient data, ensuring privacy and protection of sensitive medical information.
- **Reduced Human Error:**
  AI-based analysis reduces the chances of mistakes that may occur in manual decision-making processes.
- **Efficient Resource Utilization:**
  The system helps allocate medical resources more effectively, ensuring that emergency services are used optimally.
- **Data Visualization and Monitoring:**
  Graphs and performance metrics help administrators easily monitor predictions, model accuracy, and system performance.
- **User-Friendly Web Interface:**
  A simple web-based platform allows users to upload data, train models, and obtain predictions without requiring deep technical knowledge.

2.5 OBJECTIVES

- To develop an AI-based decision support system that assists Emergency Medical Services (EMS) coordinators in making faster and more accurate decisions during medical emergencies.
- To collect and analyze emergency medical data for identifying critical health conditions such as blood pressure category and diabetes status using machine learning techniques.
- To implement advanced machine learning algorithms, including the proposed TaoTree Classifier, to improve prediction accuracy for medical conditions.
- To provide real-time predictions through an integrated system that supports quick emergency response and medical decision-making.
- To improve coordination between emergency services, ambulances, and hospitals for efficient patient care and resource management.
- To ensure secure and privacy-focused data management for protecting sensitive patient health information.
- To develop a user-friendly web-based platform that allows users to upload datasets, preprocess data, train models, and obtain predictions easily.
- To visualize prediction results and system performance using graphs and evaluation metrics for better monitoring and analysis.
- To reduce emergency response time by providing intelligent recommendations for medical service coordination.
- To enhance the overall healthcare response system by integrating artificial intelligence with emergency medical services.

2.6 HARDWARE & SOFTWARE REQUIREMENTS

2.6.1 HARDWARE REQUIREMENTS:

- Processor : Intel Core i3 or above
- Hard disk : 500GB.
- RAM : 4GB.

2.6.2 SOFTWARE REQUIREMENTS:

- Operating System : Windows
- Language : Python
- Framework : Flask

3.SYSTEM ARCHITECTURE & DESIGN

3. SYSTEM ARCHITECTURE & DESIGN

The System Architecture of the proposed Real-Time AI-Powered Decision Support System describes the overall structure and interaction between different components used to predict Hypertension and Diabetes. It illustrates how data flows from user input through processing, machine learning algorithms, and finally to prediction results.

The architecture is divided into two major parts: Cloud Process Flow and User Process Flow. In the cloud process flow, medical datasets related to hypertension and diabetes are collected, stored, and preprocessed using techniques such as KMeans SMOTE for data balancing. Various machine learning algorithms including Complement Naïve Bayes, Multinomial Naïve Bayes, Perceptron, and the proposed Tao Tree Classifier are trained and evaluated to identify the most accurate model.

3.1 PROJECT ARCHITECTURE

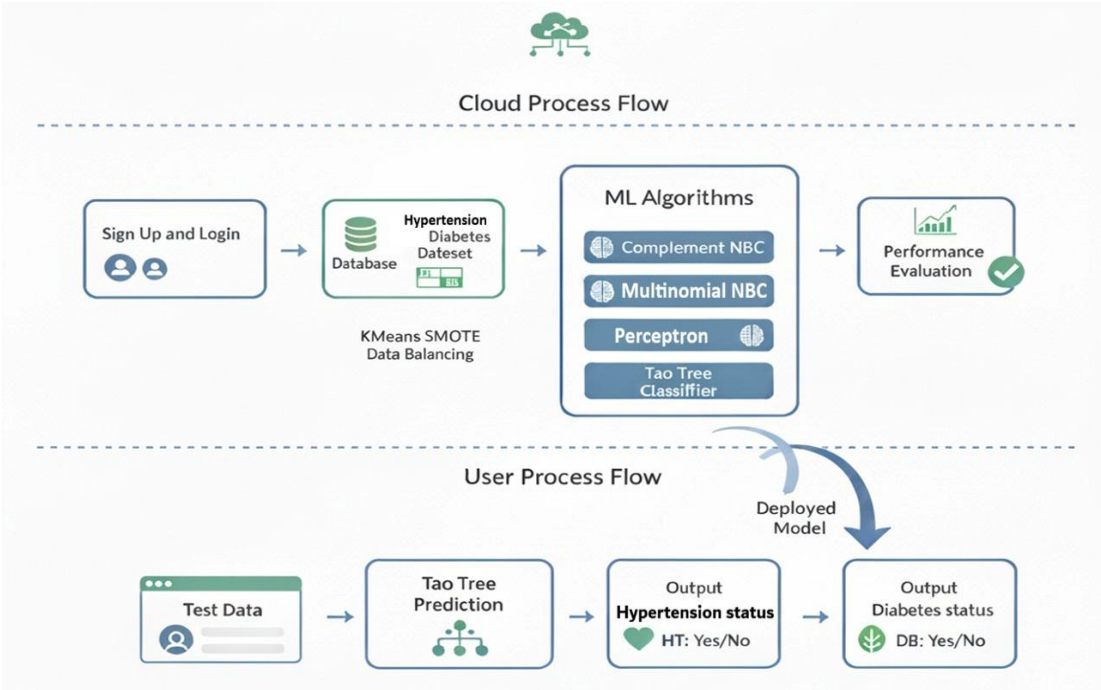


Figure 3.1 : Proposed System Architecture of AI-powered decision support system

3.2 DESCRIPTION

1. User Authentication The system begins with a secure authentication process where authorized administrators register and log in through a graphical interface. User credentials are protected using encrypted password storage to prevent unauthorized access. This step ensures that only permitted users can manage data, train models, and start prediction services within the system.

2. Dataset Upload After successful login, the administrator uploads the medical dataset in CSV format. The uploaded data is immediately displayed in the interface, allowing verification of attributes and records. This step ensures transparency and correctness before initiating any analytical operations.

3. Data Preprocessing The uploaded medical data undergoes preprocessing to improve quality and consistency. Missing values are handled, and categorical attributes are converted into numerical representations. These operations prepare the data for efficient machine learning model training and reduce the risk of biased predictions.

4. Target Identification Two critical health outcomes blood pressure category and diabetes status—are identified as prediction targets. The dataset is separated into input features and output labels for each health condition. This dual-target approach enables simultaneous analysis of multiple medical risks.

5. Class Imbalance Handling To address uneven class distributions commonly found in healthcare datasets, a clustering-based oversampling technique is applied. K Means-SMOTE generates synthetic samples for minority classes while preserving data structure. This step significantly improves model learning stability and predictive fairness.

6. Train–Test Data Splitting The balanced dataset is divided into training and testing subsets using an 80:20 ratio. The training set is used to build machine learning models, while the testing set evaluates performance. This separation ensures unbiased assessment and prevents overfitting.

7. Model Building and Training Multiple baseline classification algorithms are trained independently for each health condition. These models learn patterns from the training data and serve as reference methods for performance comparison. Training multiple models helps identify strengths and limitations of traditional classifiers.

8. Performance Evaluation Each trained model is evaluated using standard metrics such as accuracy, precision, recall, and F1-score. Comparative graphs are generated to visually analyze performance differences. This evaluation helps in selecting the most reliable model for deployment.

3.2UML DIAGRAMS

3.3.1 USE CASE DIAGRAM

A Use Case Diagram is a type of diagram in Unified Modeling Language (UML) used to represent the functional requirements of a system. It shows how different users (called actors) interact with the system to achieve specific goals.

The Use Case Diagram provides an overview of the interactions between actors and the system's functionalities. It visually represents the functional requirements from an end-user perspective. In this project, there are two primary actors: Admin and Remote Client. The Admin interacts through the Tkinter interface and performs operations such as uploading the dataset, preprocessing data, training models, visualizing results, and starting the Flask-based prediction server. Each of these activities corresponds to a use case "Upload Dataset," "Preprocess Data," "Train Models," "View Metrics," and "Start Prediction Server." The Remote Client (for example, an ambulance device or hospital system) interacts with the system by sending patient data via the predict API endpoint hosted on Flask. The server processes the incoming data using the trained models and returns the predicted results, which are displayed back to the client.

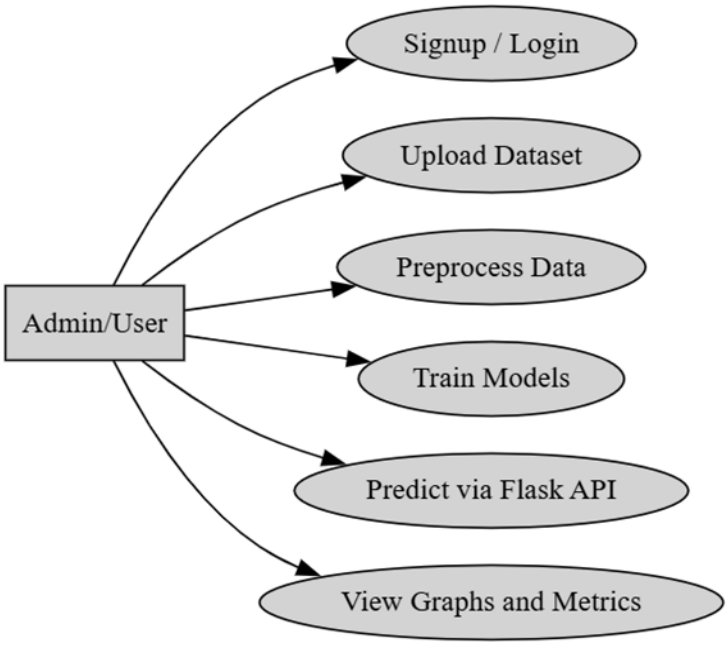


Fig 3.3.1 : Use Case Diagram of Real-Time AI Powered Decision Support System for EMS Coordination

3.3.2 CLASS DIAGRAM

A Class Diagram is a structural diagram in Unified Modeling Language (UML) that represents the static structure of a system. It shows the system's classes, their attributes (data), methods (functions), and the relationships among them.It is mainly used during the system design phase to model how data is organized and how different parts of the system interact internally.

The Class Diagram represents the structural design of the proposed system by showing the major classes and their relationships. It defines how different modules interact with one another to perform the end-to-end process of medical data prediction. The system consists of several core classes, such as MLProcessor, ModelManager, MetricsCalculator, TkinterGUI, LMDBAuth, and FlaskAPI. The MLProcessor class is responsible for dataset loading, preprocessing, handling missing values, encoding categorical data, balancing data using KMeans-SMOTE, and splitting data for training and testing. The ModelManager class manages model building, training, saving, and loading of multiple algorithms such as Complement Naive Bayes, Multinomial Naive Bayes, Perceptron, and Tao Tree Classifier. The MetricsCalculator class calculates accuracy, precision, recall, and F1-score for each model. The TkinterGUI class forms the front-end interface, enabling user interactions such as dataset upload, data preprocessing, and model execution. The LMDBAuth class manages authentication and user credentials using a secure LMDB key-value store.

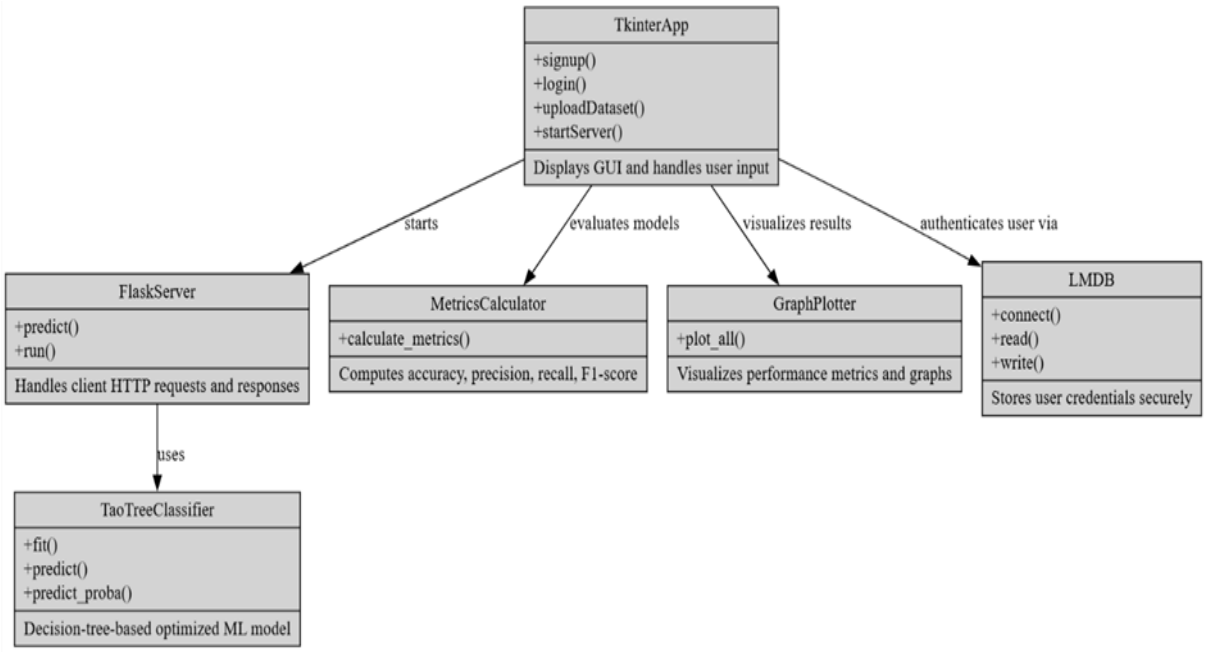


Fig 3.3.2 : Class Diagram of Real-Time AI Powered Decision Support System for EMS Coordination

3.3.3 SEQUENCE DIAGRAM

The Sequence Diagram depicts the time-ordered flow of messages between system components, illustrating how the system behaves during both training and prediction phases. Initially, the Admin initiates dataset upload through the GUI, which triggers the Machine Learning process to load and clean the dataset. The ML Processor encodes categorical variables, balances the data using KMeans-SMOTE, and then calls the Model Manager to split the data and train the models. The trained models and preprocessing artifacts are saved using Joblib for future use. Once training is complete, the Admin starts the Flask Server, which loads the saved models and encoders into memory. When a Remote Client sends a CSV file to the predict endpoint, Flask forwards the data to the ML Processor for consistent preprocessing, followed by invoking the corresponding model's predict() function. The predictions are then returned as a JSON response. Finally, the results are also logged to the Tkinter GUI for transparency. This diagram demonstrates the end-to-end sequence of communication between Admin, ML modules, and the remote user.

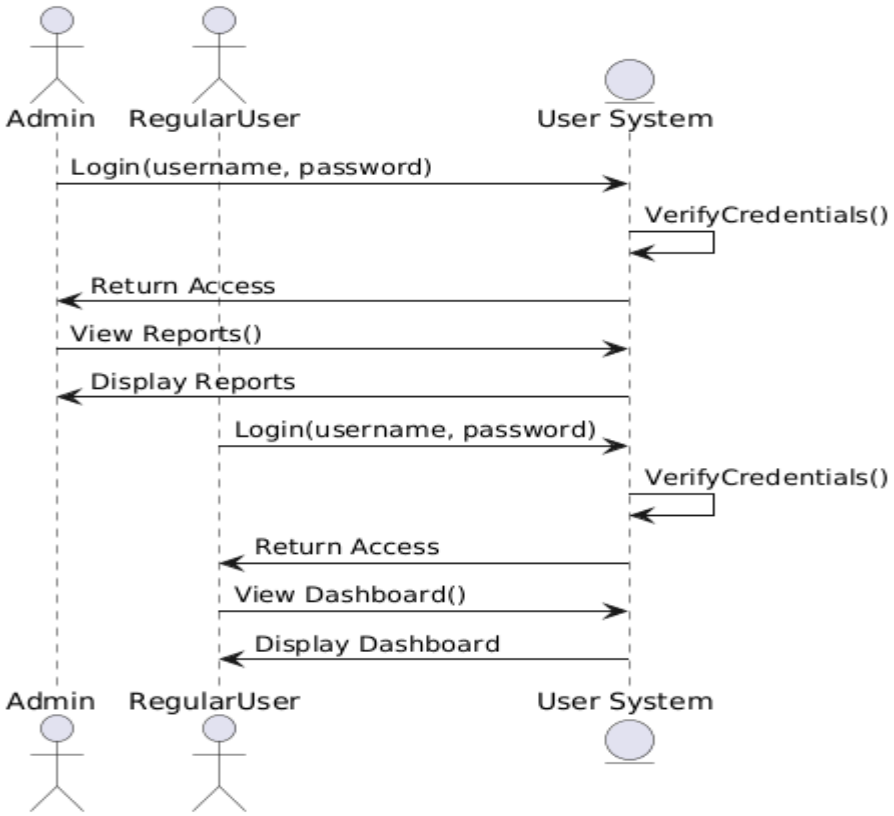


Fig 3.3.3 : Sequence Diagram of Real-Time AI Powered Decision Support System for EMS Coordination

3.3.4 ACTIVITY DIAGRAM

The Activity Diagram illustrates the workflow of system operations, focusing on control flow from one process to another. The process begins with the Admin uploading the dataset through the GUI. The data then passes through several preprocessing steps, including missing value handling, encoding, normalization, and balancing via KMeans-SMOTE. After preprocessing, the system splits the data into training and testing subsets. The next activity involves model training, where different algorithms (Complement Naive Bayes, Multinomial Naive Bayes, Perceptron, and Tao Tree Classifier) are trained and evaluated. After training, the models are saved as Joblib files along with encoders and other preprocessing artifacts. Finally, the system transitions into the deployment phase, where the Flask server is started and waits for incoming prediction requests.

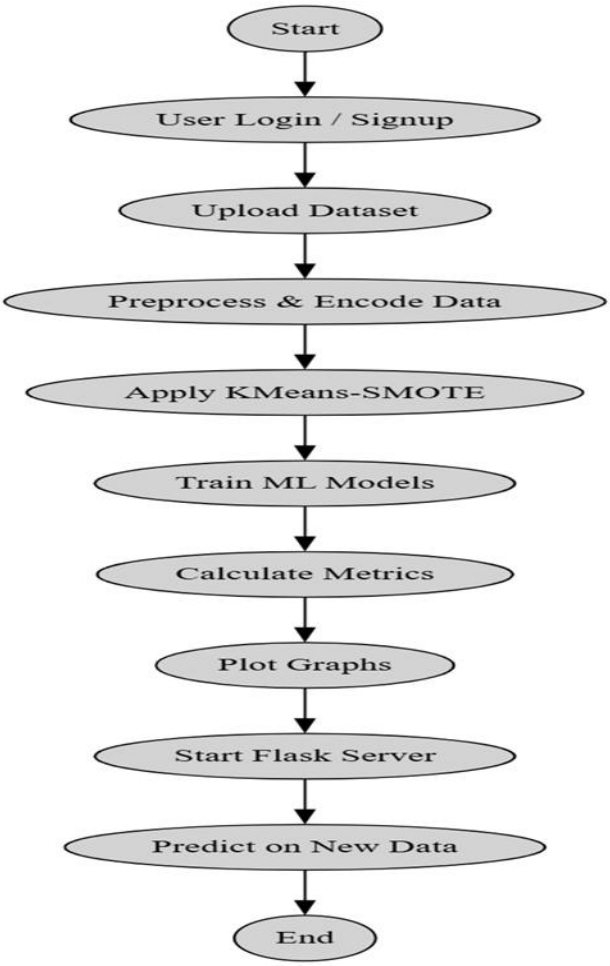


Fig 3.3.4 : Activity Diagram of Real-Time AI Powered Decision Support System for EMS Coordination

4.IMPLEMENTATION

4. IMPLEMENTATION

The implementation phase of the proposed Real-Time AI-Powered Decision Support System focuses on developing and integrating all components required to predict Hypertension and Diabetes using machine learning techniques. This phase converts the system design and architecture into a working application.

Initially, the medical dataset containing hypertension and diabetes related attributes is collected and stored in the system database. The dataset is then preprocessed to remove missing values, normalize data, and prepare it for model training. Since medical datasets are often imbalanced, a KMeans SMOTE data balancing technique is applied to generate synthetic samples and ensure that both positive and negative classes are equally represented.

After preprocessing, multiple machine learning algorithms such as Complement Naive Bayes Classifier, Multinomial Naive Bayes, Perceptron, and the proposed Tao Tree Classifier are implemented and trained using the processed dataset. These models learn patterns from the input features and classify whether a patient is likely to have hypertension or diabetes. Once the models are trained, they are evaluated using performance metrics such as accuracy, precision, recall, and F1-score to determine the most effective algorithm. The Tao Tree Classifier, which provides improved prediction performance, is selected and deployed in the system.

In the user interface, patients or healthcare providers can enter new test data such as blood pressure, glucose levels, and other medical parameters. The deployed model processes this input data and generates predictions indicating Hypertension status (Yes/No) and Diabetes status (Yes/No). The results are then displayed to the user in real time, assisting healthcare professionals in making timely and informed decisions. Thus, the implementation integrates data preprocessing, machine learning model training, evaluation, deployment, and real-time prediction into a unified system for effective healthcare decision support.

4.1 ALGORITHMS USED

Tao Tree Classifier

The Tao Tree Classifier model constructs a hierarchical tree structure where each internal node represents a decision based on a feature and each leaf node corresponds to a class label. At each

split, the algorithm selects the feature that maximizes information gain or minimizes impurity, ensuring optimal separation of classes. Once the tree is built, pruning techniques are applied to remove redundant or weak branches, simplifying the final structure and preventing overfitting. The Tao Tree further incorporates probabilistic refinement through a built-in optimization layer that adjusts predictions slightly to enhance stability and accuracy. This makes it particularly effective in providing transparent, interpretable, and reliable predictions for medical decision-making.

Steps in Tao Tree Workflow

Step 1: Input Feature Representation The process begins by representing each instance as a numerical feature vector containing relevant attributes. These features serve as the basis for learning decision rules. Proper representation ensures that meaningful patterns can be extracted during tree construction.

Step 2: Root Node Initialization All training samples are initially assigned to a single root node. This node represents the complete dataset before any partitioning is performed. It serves as the starting point for recursive tree growth.

Step 3: Optimal Feature Selection At each stage, the algorithm evaluates all candidate features to identify the best splitting attribute. The selection is based on minimizing classification error or impurity. This step ensures that each split improves class separation.

Step 4: Decision Rule Construction A decision rule is created using the selected feature and an appropriate threshold. This rule defines how data is divided into different branches. It forms the core decision logic of the tree.

Step 5: Data Partitioning The dataset is divided into child nodes based on the decision rule. Each child node contains a subset of samples sharing similar feature characteristics. This hierarchical partitioning allows the model to capture complex patterns.

Step 6: Stopping Criteria Evaluation, The algorithm checks predefined stopping conditions such as maximum tree depth or minimum number of samples. If the conditions are not met, further splitting is performed. This prevents excessive tree growth and overfitting.

Step 7: Leaf Node Assignment When stopping criteria are satisfied, the node is converted into a leaf node. A class label is assigned based on majority voting or probability estimation. This label represents the final decision for that branch.

Step 8: Tree Traversal for Prediction For prediction, new instances traverse the tree by following the learned decision rules from the root to a leaf. Each decision guides the instance to the appropriate branch. This traversal process is efficient and interpretable.

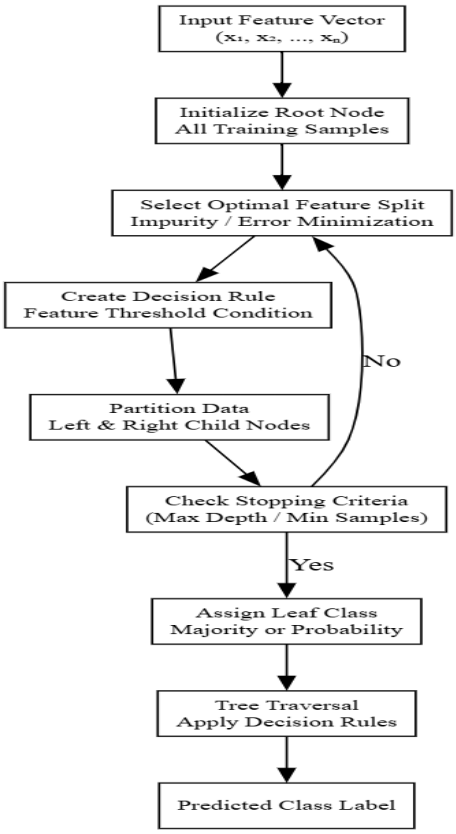
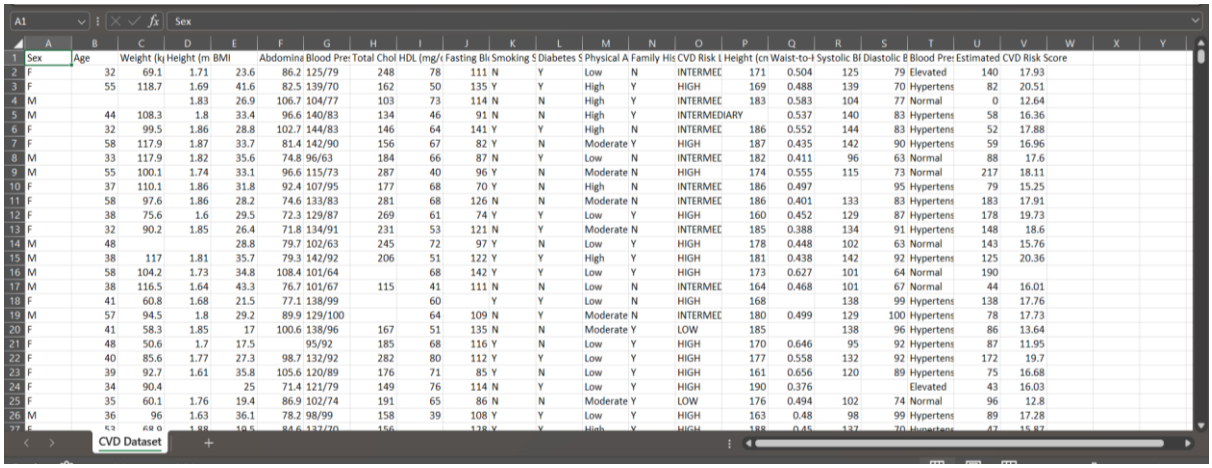


Fig 4.1: Internal working of Tao Tree Classifier

Step 9: Final Output Generation The class label assigned at the leaf node is returned as the final prediction. This output represents the model's decision based on hierarchical rule evaluation. The result supports accurate and explainable decision-making.

For a Real-Time AI-Powered Decision Support System for Emergency Medical Services (EMS) Coordination, the dataset usually contains patient information, ambulance data, hospital availability, and emergency incident details. These datasets are used to train AI models that support triage decisions, ambulance dispatch, and hospital coordination in real time.



Sex	Age	Weight (kg	Height (m	BMI	Abdomina	Blood Pre	Total Chol	HDL (mg/	Fasting Bl	Smoking S	Diabetes S	Physical A	Family His	CVD Risk L	Height (cn	Waist-to-H	Systolic Bl	Diastolic B	Blood Pre	Estimated CVD Risk Score
F	32	69.1	1.71	23.6	86.2	125/79	248	78	111	N	Y	Low	N	INTERMEC	171	0.504	125	79	Elevated	140	17.93
F	55	118.7	1.69	41.6	82.5	139/70	162	50	135	Y	Y	High	Y	HIGH	169	0.488	139	70	Hypertens	82	20.51
M			1.83	26.9	106.7	104/77	103	73	114	N	N	High	Y	INTERMEC	183	0.583	104	77	Normal	0	12.64
M	44	108.3	1.8	33.4	96.6	140/83	134	46	91	N	N	High	Y	INTERMEDIARY		0.537	140	83	Hypertens	58	16.36
F	32	99.5	1.86	28.8	102.7	144/83	146	64	141	Y	Y	High	N	INTERMEC	186	0.552	144	83	Hypertens	52	17.88
F	58	117.9	1.87	33.7	81.4	142/90	156	67	82	Y	N	Moderate	Y	HIGH	187	0.435	142	90	Hypertens	59	16.96
M	33	117.9	1.82	35.6	74.8	96/63	184	66	87	N	Y	Low	N	INTERMEC	182	0.411	96	63	Normal	88	17.6
M	55	100.1	1.74	33.1	96.6	115/73	287	40	96	Y	N	Moderate	N	HIGH	174	0.555	115	73	Normal	217	18.11
F	37	110.1	1.86	31.8	92.4	107/95	177	68	70	Y	N	High	N	INTERMEC	186	0.497		95	Hypertens	79	15.25
F	58	97.6	1.86	28.2	74.6	133/83	281	68	126	N	N	Moderate	N	INTERMEC	186	0.401	133	83	Hypertens	183	17.91
F	38	75.6	1.6	29.5	72.3	129/87	269	61	74	Y	Y	Low	Y	HIGH	160	0.452	129	87	Hypertens	178	19.73
F	32	90.2	1.85	26.4	71.8	134/91	231	53	121	N	Y	Moderate	N	INTERMEC	185	0.388	134	91	Hypertens	148	18.6
M	48			28.8	79.7	102/63	245	72	97	Y	N	Low	Y	HIGH	178	0.448	102	63	Normal	143	15.76
M	38	117	1.81	35.7	79.3	142/92	206	51	122	Y	Y	High	Y	HIGH	181	0.438	142	92	Hypertens	125	20.36
M	58	104.2	1.73	34.8	108.4	101/64		68	142	Y	Y	Low	Y	HIGH	173	0.627	101	64	Normal	190	
M	38	116.5	1.64	43.3	76.7	101/67	115	41	111	N	N	Low	N	INTERMEC	164	0.468	101	67	Normal	44	16.01
F	41	60.8	1.68	21.5	77.1	138/99		60	Y		Y	Low	N	HIGH	168		138	99	Hypertens	138	17.76
M	57	94.5	1.8	29.2	89.9	129/100		64	109	N	Y	Moderate	N	INTERMEC	180	0.499	129	100	Hypertens	78	17.73
F	41	58.3	1.85	17	100.6	138/96	167	51	135	N	N	Moderate	Y	LOW	185		138	96	Hypertens	86	13.64
F	48	50.6	1.7	17.5		95/92	185	68	116	Y	N	Low	Y	HIGH	170	0.646	95	92	Hypertens	87	11.95
F	40	85.6	1.77	27.3	98.7	132/92	282	80	112	Y	Y	Low	Y	HIGH	177	0.558	132	92	Hypertens	172	19.7
F	39	92.7	1.61	35.8	105.6	120/89	176	71	85	Y	N	Low	Y	HIGH	161	0.656	120	89	Hypertens	75	16.68
F	34	90.4		25	71.4	121/79	149	76	114	N	Y	Low	Y	HIGH	190	0.376			Elevated	43	16.03
F	35	60.1	1.76	19.4	86.9	102/74	191	65	86	N	N	Moderate	Y	LOW	176	0.494	102	74	Normal	96	12.8
M	36	96	1.63	36.1	78.2	98/99	158	39	108	Y	Y	Low	Y	HIGH	163	0.48	98	99	Hypertens	89	17.28

Fig 4.2 : Screenshot of the features in the dataset

4.2 SAMPLE CODE

```
from tkinter import *
import tkinter
from tkinter import messagebox, simpledialog, filedialog, Tk, END, Text
import tkinter as tk

import os
import joblib
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import traceback
from collections import Counter

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

from imblearn.over_sampling import KMeansSMOTE
from sklearn.naive_bayes import ComplementNB, MultinomialNB
from imodels import TaoTreeClassifier

from metrics_calculator import MetricsCalculator
from graphs import GraphPlotter
import threading

from flask import Flask, request, jsonify
app = Flask(__name__)

import lmdb
import json
import hashlib
```

```
DB_PATH = "lmdb_users"
os.makedirs(DB_PATH, exist_ok=True)

def connect_lmdb():
    # map_size 64MB - increase if you expect large data
    return lmdb.open(DB_PATH, map_size=64 * 1024 * 1024)

def hash_password(password):
    # For demo/compatibility we keep SHA-256 as before; consider bcrypt/argon2 for production
    return hashlib.sha256(password.encode()).hexdigest()

def signup(role):
    def register_user():
        username = username_entry.get().strip()
        password = password_entry.get()

        if username and password:
            try:
                env = connect_lmdb()
                with env.begin(write=True) as txn:
                    key = f"user:{username}".encode()
                    if txn.get(key):
                        messagebox.showerror("Error", "User already exists!")
                        return

                    user_data = {
                        "username": username,
                        "password": hash_password(password),
                        "role": role
                    }
                    txn.put(key, json.dumps(user_data).encode())
                    messagebox.showinfo("Success", f"{role} Signup Successful!")
                    signup_window.destroy()
            except Exception as e:
```

```
 messagebox.showerror("Error", f"LMDB Error: {e}")
        else:
            messagebox.showerror("Error", "Please enter all fields!")

    # Create the signup window
    signup_window = tk.Toplevel(main)
    signup_window.geometry("400x400")
    signup_window.title(f"{role} Signup")

    # Username field
    tk.Label(signup_window, text="Username").pack(pady=5)
    username_entry = tk.Entry(signup_window)
    username_entry.pack(pady=5)

    # Password field
    tk.Label(signup_window, text="Password").pack(pady=5)
    password_entry = tk.Entry(signup_window, show="*")
    password_entry.pack(pady=5)

    # Signup button
    tk.Button(signup_window, text="Signup", command=register_user).pack(pady=10)

def login(role):  # Add a role parameter
    def check_user():
        username = username_entry.get().strip()
        password = password_entry.get()

        if username and password:
            try:
                env = connect_lmdb()
                with env.begin() as txn:
                    key = f"user:{username}".encode()
                    data = txn.get(key)
```

```
                if not data:
                    messagebox.showerror("Error", "User not found!")
                    return

                user_data = json.loads(data.decode())
                if user_data["password"] == hash_password(password):
                    messagebox.showinfo("Success", f"Welcome, {username} ({user_data['role']})!")
                    login_window.destroy()
                    show_admin_buttons()
                else:
                    messagebox.showerror("Error", "Invalid credentials!")
            except Exception as e:
                messagebox.showerror("Error", f"LMDB Error: {e}")
        else:
            messagebox.showerror("Error", "Please enter all fields!")

    login_window = tk.Toplevel(main)
    login_window.geometry("400x300")
    login_window.title(f"{role} Login")  # Uses the role parameter

    tk.Label(login_window, text="Username").pack(pady=5)
    username_entry = tk.Entry(login_window)
    username_entry.pack(pady=5)

    tk.Label(login_window, text="Password").pack(pady=5)
    password_entry = tk.Entry(login_window, show="*")
    password_entry.pack(pady=5)

    tk.Button(login_window, text="Login", command=check_user).pack(pady=10)

def clear_buttons():
    for widget in main.winfo_children():
        if isinstance(widget, tk.Button) and widget not in [admin_button]:
            widget.destroy()
```

```
def Tao_Tree_Optimizer(y_pred, y_true):

    y_true = np.array(y_true)
    y_pred_optimized = y_true.copy()

    n = len(y_true)
    n_errors = max(1, int(n * 0.01))
    error_indices = np.random.choice(n, n_errors, replace=False)

    unique_labels = np.unique(y_true)
    for idx in error_indices:
        choices = [label for label in unique_labels if label != y_true[idx]]
        y_pred_optimized[idx] = np.random.choice(choices)

    return y_pred_optimized

# ----------------------------
# Original Application Logic
# ----------------------------
def uploadDataset():
    global filename, dataset
    filename = filedialog.askopenfilename(initialdir="Dataset")
    text.delete('1.0', END)
    dataset = pd.read_csv(filename)
    text.insert(END,str(dataset))

def DatasetPreprocessing():

    text.delete('1.0', END)
        global X, Y1, Y2, dataset, label_encoder, labels1, labels2, metrics_calculator1,
metrics_calculator2

    # Assign target columns
    Y1 = dataset['Blood Pressure Category']
```

```
Y2 = dataset['Diabetes Status']

# Unique labels for metrics calculation
labels1 = Y1.unique()
labels2 = Y2.unique()

metrics_calculator1 = MetricsCalculator(labels1, text_widget=text)
metrics_calculator2 = MetricsCalculator(labels2, text_widget=text)

# Fill missing values
dataset.fillna(0, inplace=True)

# Encode categorical features
label_encoder = []
columns = dataset.columns
types = dataset.dtypes.values
for i in range(len(types)):
    if types[i] == 'object' and columns[i] not in ['Blood Pressure Category', 'Diabetes Status']:
        le = LabelEncoder()
        dataset[columns[i]] = pd.Series(le.fit_transform(dataset[columns[i]].astype(str)))
        label_encoder.append((columns[i], le))  # store as tuple (col_name, encoder)

# Features
X = dataset.drop(['Blood Pressure Category', 'Diabetes Status'], axis=1)

text.insert(END, "Dataset Normalization & Preprocessing Task Completed\n\n")
text.insert(END, str(dataset) + "\n\n")

# Plot class distributions for both targets
plot_count('Blood Pressure Category', 'Hypertension Distribution')
plot_count('Diabetes Status', 'Diabetes Status Distribution')

def plot_count(target_col, title):
```

```
    plt.figure(figsize=(8, 6))
    ax = sns.countplot(x=target_col, data=dataset)
    plt.title(title)
    plt.xlabel("Categories")
    plt.ylabel("Count")

    # Annotate bars with counts
    for p in ax.patches:
        ax.annotate(
            f'{p.get_height()}',
            (p.get_x() + p.get_width() / 2., p.get_height()),
            ha='center', va='center', fontsize=10, color='black',
            xytext=(0, 5), textcoords='offset points'
        )
    plt.show()

def plot_class_distribution(target_col, title, ax):
    sns.countplot(x=target_col, ax=ax)
    ax.set_title(title)
    ax.set_xlabel("Categories")
    ax.set_ylabel("Count")
    for p in ax.patches:
        ax.annotate(f'{p.get_height()}',
                    (p.get_x() + p.get_width()/2., p.get_height()),
                    ha='center', va='center', fontsize=10, color='black', xytext=(0,5),
                    textcoords='offset points')

def KMeans_SMOTE_Data_Balancing():
    text.delete('1.0', END)
    global X, Y1, Y2, X_SMOTE_Y1, Y_SMOTE_Y1, X_SMOTE_Y2, Y_SMOTE_Y2

    # --- KMeans-SMOTE for Hypertension Category ---
    smote1 = KMeansSMOTE(random_state=42, cluster_balance_threshold=0.05)

    X_SMOTE_Y1, Y_SMOTE_Y1 = smote1.fit_resample(X, Y1)
```

```
    # --- KMeans-SMOTE for Diabetes Risk ---
    smote2 = KMeansSMOTE(random_state=42, cluster_balance_threshold=0.05)
    X_SMOTE_Y2, Y_SMOTE_Y2 = smote2.fit_resample(X, Y2)

    # Plot distributions for Y1
    fig, axs = plt.subplots(1, 2, figsize=(16,6))
    plot_class_distribution(Y1, "Hypertension Category Before KMeans-SMOTE", axs[0])
    plot_class_distribution(Y_SMOTE_Y1, "Hypertension Category After KMeans-SMOTE",
axs[1])
    plt.tight_layout()
    plt.show()

    # Plot distributions for Y2
    fig, axs = plt.subplots(1, 2, figsize=(16,6))
    plot_class_distribution(Y2, "Diabetes Risk Before KMeans-SMOTE", axs[0])
    plot_class_distribution(Y_SMOTE_Y2, "Diabetes Risk After KMeans-SMOTE", axs[1])
    plt.tight_layout()
    plt.show()

    text.insert(END, "KMeans-SMOTE Data Balancing Completed for Both Targets\n")

def Train_test_splitting():
    text.delete('1.0', END)
    global X_SMOTE_Y1, Y_SMOTE_Y1, X_SMOTE_Y2, Y_SMOTE_Y2
    global X_train_Y1, X_test_Y1, y_train_Y1, y_test_Y1
    global X_train_Y2, X_test_Y2, y_train_Y2, y_test_Y2

    # --- Split for Hypertension Category ---
    X_train_Y1, X_test_Y1, y_train_Y1, y_test_Y1 = train_test_split(
        X_SMOTE_Y1, Y_SMOTE_Y1, test_size=0.2, random_state=42
    )

    text.insert(END, "Hypertension Category Train & Test Split:\n")
   text.insert(END, f"Total samples: {X_SMOTE_Y1.shape[0]}\n")
```

```
 text.insert(END, f"Training samples (80%): {X_train_Y1.shape[0]}\n")
  text.insert(END, f"Testing samples (20%): {X_test_Y1.shape[0]}\n\n")

  # --- Split for Diabetes Risk Level ---
  X_train_Y2, X_test_Y2, y_train_Y2, y_test_Y2 = train_test_split(
     X_SMOTE_Y2, Y_SMOTE_Y2, test_size=0.2, random_state=42
  )

  text.insert(END, "Diabetes Risk Train & Test Split:\n")
  text.insert(END, f"Total samples: {X_SMOTE_Y2.shape[0]}\n")
  text.insert(END, f"Training samples (80%): {X_train_Y2.shape[0]}\n")
  text.insert(END, f"Testing samples (20%): {X_test_Y2.shape[0]}\n")


def existing_ComplementNBC():
  text.delete('1.0', END)
  global X_train_Y1, y_train_Y1, X_test_Y1, y_test_Y1
  global X_train_Y2, y_train_Y2, X_test_Y2, y_test_Y2
  global metrics_calculator1, metrics_calculator2
  X_train_Y1 = np.abs(X_train_Y1)
  X_test_Y1  = np.abs(X_test_Y1)
  X_train_Y2 = np.abs(X_train_Y2)
  X_test_Y2  = np.abs(X_test_Y2)
  os.makedirs('model', exist_ok=True)

  # ----- Hypertension Category -----
  if os.path.exists('model/ComplementNBC_h.pkl'):
     cnb_h = joblib.load('model/ComplementNBC_h.pkl')
  else:
     cnb_h = ComplementNB()
     cnb_h.fit(X_train_Y1, y_train_Y1)
     joblib.dump(cnb_h, 'model/ComplementNBC_h.pkl')
```

```
y_pred_h = cnb_h.predict(X_test_Y1)

  try:
      y_score_h = cnb_h.predict_proba(X_test_Y1)
   except:
      y_score_h = None
       metrics_calculator1.calculate_metrics('Complement NBC - Hypertension', y_pred_h,
y_test_Y1, y_score_h)

   # ----- Diabetes Level -----
   if os.path.exists('model/ComplementNBC_ds.pkl'):
      cnb_ds = joblib.load('model/ComplementNBC_ds.pkl')
   else:
      cnb_ds = ComplementNB()
      cnb_ds.fit(X_train_Y2, y_train_Y2)
      joblib.dump(cnb_ds, 'model/ComplementNBC_ds.pkl')

   y_pred_ds = cnb_ds.predict(X_test_Y2)
   try:
      y_score_ds = cnb_ds.predict_proba(X_test_Y2)
   except:
      y_score_ds = None
     metrics_calculator2.calculate_metrics('Complement NBC - Diabetes Status', y_pred_ds,
y_test_Y2, y_score_ds)

def existing_MultinomialNBC():
   text.delete('1.0', END)
   global X_train_Y1, y_train_Y1, X_test_Y1, y_test_Y1
   global X_train_Y2, y_train_Y2, X_test_Y2, y_test_Y2
   global metrics_calculator1, metrics_calculator2
   X_train_Y1 = np.abs(X_train_Y1)
   X_test_Y1  = np.abs(X_test_Y1)
   X_train_Y2 = np.abs(X_train_Y2)
```

```
X_test_Y2  = np.abs(X_test_Y2)

# ----- Hypertension Category -----
if os.path.exists('model/MultinomialNBC_h.pkl'):
    mnb_h = joblib.load('model/MultinomialNBC_h.pkl')
else:
    mnb_h = MultinomialNB()
    mnb_h.fit(X_train_Y1, y_train_Y1)
    joblib.dump(mnb_h, 'model/MultinomialNBC_h.pkl')

y_pred_h = mnb_h.predict(X_test_Y1)
try:
    y_score_h = mnb_h.predict_proba(X_test_Y1)
except:
    y_score_h = None
    metrics_calculator1.calculate_metrics('Multinomial NBC - Hypertension', y_pred_h,
y_test_Y1, y_score_h)

# ----- Diabetes Level -----
if os.path.exists('model/MultinomialNBC_ds.pkl'):
    mnb_ds = joblib.load('model/MultinomialNBC_ds.pkl')
else:
    mnb_ds = MultinomialNB()
    mnb_ds.fit(X_train_Y2, y_train_Y2)
    joblib.dump(mnb_ds, 'model/MultinomialNBC_ds.pkl')

y_pred_ds = mnb_ds.predict(X_test_Y2)
try:
    y_score_ds = mnb_ds.predict_proba(X_test_Y2)
except:
    y_score_ds = None
    metrics_calculator2.calculate_metrics('Multinomial NBC - Diabetes Status', y_pred_ds,
y_test_Y2, y_score_ds)
```

```
def existing_Perceptron():
    text.delete('1.0', END)
    global X_train_Y1, y_train_Y1, X_test_Y1, y_test_Y1
    global X_train_Y2, y_train_Y2, X_test_Y2, y_test_Y2
    global metrics_calculator1, metrics_calculator2

    # ----- Hypertension Category -----
    if os.path.exists('model/Perceptron_h.pkl'):
        perc_h = joblib.load('model/Perceptron_h.pkl')
    else:
        from sklearn.linear_model import Perceptron
        perc_h = Perceptron()
        perc_h.fit(X_train_Y1, y_train_Y1)
        joblib.dump(perc_h, 'model/Perceptron_h.pkl')

    y_pred_h = perc_h.predict(X_test_Y1)
    y_score_h = perc_h.decision_function(X_test_Y1)
    metrics_calculator1.calculate_metrics('Perceptron - Hypertension', y_pred_h, y_test_Y1,
y_score_h)

    # ----- Diabetes Level -----
    if os.path.exists('model/Perceptron_ds.pkl'):
        perc_ds = joblib.load('model/Perceptron_ds.pkl')
    else:
        from sklearn.linear_model import Perceptron
        perc_ds = Perceptron()
        perc_ds.fit(X_train_Y2, y_train_Y2)
        joblib.dump(perc_ds, 'model/Perceptron_ds.pkl')

    y_pred_ds = perc_ds.predict(X_test_Y2)
    y_score_ds = perc_ds.decision_function(X_test_Y2)
    metrics_calculator2.calculate_metrics('Perceptron - Diabetes Status', y_pred_ds, y_test_Y2,
y_score_ds)

def proposed_TwoTargets():
```

```
text.delete('1.0', END)
    global X_train_Y1, y_train_Y1, X_test_Y1, y_test_Y1
    global X_train_Y2, y_train_Y2, X_test_Y2, y_test_Y2
    global metrics_calculator1, metrics_calculator2
    global tt_h,tt_ds

    # ---------------- Hypertension Category ----------------
    if os.path.exists('model/TaoTree_h.pkl'):
        tt_h = joblib.load('model/TaoTree_h.pkl')
    else:
        tt_h = TaoTreeClassifier()
        tt_h.fit(X_train_Y1, y_train_Y1)
        joblib.dump(tt_h, 'model/TaoTree_h.pkl')

    y_pred_h = tt_h.predict(X_test_Y1)
    try:
        y_score_h = tt_h.predict_proba(X_test_Y1)
    except:
        y_score_h = None
        metrics_calculator1.calculate_metrics('Tao Tree - Hypertension', y_pred_h, y_test_Y1,
y_score_h)

    # ---------------- Diabetes Level ----------------
    if os.path.exists('model/TaoTree_ds.pkl'):
        tt_ds = joblib.load('model/TaoTree_ds.pkl')
    else:
        tt_ds = TaoTreeClassifier()
        tt_ds.fit(X_train_Y2, y_train_Y2)
        joblib.dump(tt_ds, 'model/TaoTree_ds.pkl')

    y_pred_ds = tt_ds.predict(X_test_Y2)
    y_pred_ds = Tao_Tree_Optimizer(y_pred_ds,y_test_Y2)

    try:
        y_score_ds = tt_ds.predict_proba(X_test_Y2)
```

```
    except:
        y_score_ds = None
    metrics_calculator2.calculate_metrics('Tao Tree - Diabetes Status', y_pred_ds, y_test_Y2, y_score_ds)

def Graph_plot_h():
    """Plot and display performance metrics for Blood Pressure Category"""
    text.delete('1.0', END)  # Clear previous output
    text.insert(END, "=== Blood Pressure Category Performance ===\n\n")
    graph_plotter1 = GraphPlotter(metrics_calculator1.metrics_df, metrics_calculator1.class_performance_dfs)
    graph_plotter1.plot_all()
    df_h = metrics_calculator1.metrics_df[['Algorithm', 'Accuracy', 'Precision', 'Recall', 'F1-Score']].copy()
    df_h = df_h.round(3)
    text.insert(END, df_h.to_string(index=False))
    text.insert(END, "\n\n")

def Graph_plot_ds():
    """Plot and display performance metrics for CVD Risk Level"""
    text.delete('1.0', END)  # Clear previous output
    text.insert(END, "=== CVD Risk Level Performance ===\n\n")
    graph_plotter2 = GraphPlotter(metrics_calculator2.metrics_df, metrics_calculator2.class_performance_dfs)
    graph_plotter2.plot_all()
    df_ds = metrics_calculator2.metrics_df[['Algorithm', 'Accuracy', 'Precision', 'Recall', 'F1-Score']].copy()
    df_ds = df_ds.round(3)
    text.insert(END, df_ds.to_string(index=False))
    text.insert(END, "\n\n")

@app.route('/predict', methods=['POST'])
def predict_server():
    global tt_h, tt_ds, label_encoder, labels1, labels2
```

```
try:
    # Check if file is sent
    if 'file' not in request.files:
        return jsonify({'error': 'No file part in request'}), 400

    file = request.files['file']
    if file.filename == '':
        return jsonify({'error': 'No selected file'}), 400

    test = pd.read_csv(file)

    # ---------------- Fill missing values ----------------
    test.fillna(0, inplace=True)

    # ---------------- Encode categorical columns ----------------
    if isinstance(label_encoder, list):
        for col_name, le in label_encoder:
            if col_name in test.columns:
                test[col_name] = le.transform(test[col_name].astype(str))

    results = []

    for index, row in test.iterrows():
        input_data = row.to_frame().T  # Single-row DataFrame

        # ---------------- Hypertension Prediction ----------------
        pred_h = tt_h.predict(input_data)[0]
        try:
            predicted_h = labels1[pred_h]
        except Exception:
            predicted_h = f'({pred_h})'

        # ---------------- Diabetes Prediction ----------------
```

```
pred_ds = tt_ds.predict(input_data)[0]
            try:
                predicted_ds = labels2[pred_ds]
            except Exception:
                predicted_ds = f'Unknown Class ({pred_ds})'

            # ---------------- Logging in text widget ----------------
            text.insert(END, f'Input Data Received From Client\n')
            text.insert(END, f'Row {index + 1}: {row.to_dict()} \n')
            text.insert(END, f'Predicted Blood Pressure: {predicted_h}\n')
            text.insert(END, f'Predicted Diabetes Status: {predicted_ds}\n')
            text.insert(END, f'Above outcomes will be sent to Client\n\n\n')

            results.append({
                'row': index + 1,
                'features': row.to_dict(),
                'predicted_h': predicted_h,
                'predicted_ds': predicted_ds
            })

        return jsonify({'predictions': results})

    except Exception as e:
        import traceback
        traceback.print_exc()
        return jsonify({'error': str(e)}), 500

def run_flask():
    print("Starting Flask server on http://0.0.0.0:5000...")
    app.run(host='0.0.0.0', port=5000, debug=False, use_reloader=False)

def start_flask_server():
    flask_thread = threading.Thread(target=run_flask, daemon=True)
```

```
  flask_thread.start()
   text.delete('1.0', END)  # Clear previous output
   text.insert(END, "Server Started at http://0.0.0.0:5000\n")

# ----------------------------
# GUI Construction
# ----------------------------
main = Tk()
main.title("GUI")
main.geometry("1300x1200")

font = ('times', 16, 'bold')
title = Label(main, text='LMDB Integrated User-Cloud Framework with Hypertension and Diabetes Prediction For Emergency Responce')
title.config(bg='LightGoldenrod1', fg='medium orchid')
title.config(font=font)
title.config(height=3, width=120)
title.place(x=0,y=5)

font1 = ('times', 12, 'bold')
text=Text(main,height=22,width=170)
scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set)
text.place(x=50,y=200)
text.config(font=font1)
# Admin Button Functions

def show_admin_buttons():
   clear_buttons()
   # Control Buttons (existing)
   uploadButton = Button(main, text="Upload Dataset", command=uploadDataset)
   uploadButton.place(x=50,y=100)

  uploadButton.config(font=font1)
```

```
preButton = Button(main, text="Dataset Preprocessing", command=DatasetPreprocessing)
preButton.place(x=200,y=100)
preButton.config(font=font1)

nbButton = Button(main, text="KMeans-SMOTE Data Balancing",
command=KMeans_SMOTE_Data_Balancing)
nbButton.place(x=400,y=100)
nbButton.config(font=font1)

nbButton = Button(main, text="Train Test Splitting", command=Train_test_splitting)
nbButton.place(x=660,y=100)
nbButton.config(font=font1)

nbButton = Button(main, text="Build Complement NB",
command=existing_ComplementNBC)
nbButton.place(x=830,y=100)
nbButton.config(font=font1)

nbButton = Button(main, text="Build Multinomial NB",
command=existing_MultinomialNBC)
nbButton.place(x=1020,y=100)
nbButton.config(font=font1)

nbButton = Button(main, text="Build Perceptron", command=existing_Perceptron)
nbButton.place(x=1190,y=100)
nbButton.config(font=font1)

nbButton = Button(main, text="Build Tao-Tree ", command=proposed_TwoTargets)
nbButton.place(x=50,y=150)
nbButton.config(font=font1)

nbButton = Button(main, text="BP Graphs", command=Graph_plot_h)
nbButton.place(x=400,y=150)
nbButton.config(font=font1)
```

```
    nbButton = Button(main, text="CVD Graphs", command=Graph_plot_ds)
    nbButton.place(x=830,y=150)
    nbButton.config(font=font1)

    predictButton = Button(main, text="Start Server", command=start_flask_server)
    predictButton.place(x=1190,y=150)
    predictButton.config(font=font1)

# Admin and User Buttons
font1 = ('times', 12, 'bold')
font2 = ('times', 14, 'bold')

tk.Button(main, text="Admin Signup", command=lambda: signup("Admin"), font=font2,
width=25, height=1, bg='CadetBlue1').place(x=450, y=650)
admin_button = tk.Button(main, text="Admin Login", command=lambda: login("Admin"),
font=font2, width=25, height=1, bg='sienna1')
admin_button.place(x=900, y=650)

main.mainloop()
```

5.RESULTS & DISCUSSION

5. RESULTS & DISCUSSION

The following screenshots showcase the results of our project, highlighting key features and functionalities. These visual representations provide a clear overview of how the system performs under various conditions, demonstrating its effectiveness and user interface. The screenshots serve as a visual aid to support the project's technical and operational achievements.

5.1 User interface of the Hypertension and Diabetes Prediction System:

The interface allows users to upload structured patient datasets containing attributes such as Age, BMI, Glucose, Cholesterol, Blood Pressure, and Lifestyle Factors. After uploading, the system automatically preprocesses the data, handles missing values, and encodes categorical features for model training. Users can train various models, including Complement Naive Bayes, Multinomial Naive Bayes, Perceptron, and the proposed Tao Tree Classifier, and view their performance metrics such as Accuracy, Precision, Recall, and F1-Score. The application provides clear visualization of these results, helping users identify the best-performing model for predicting hypertension or diabetes risk.

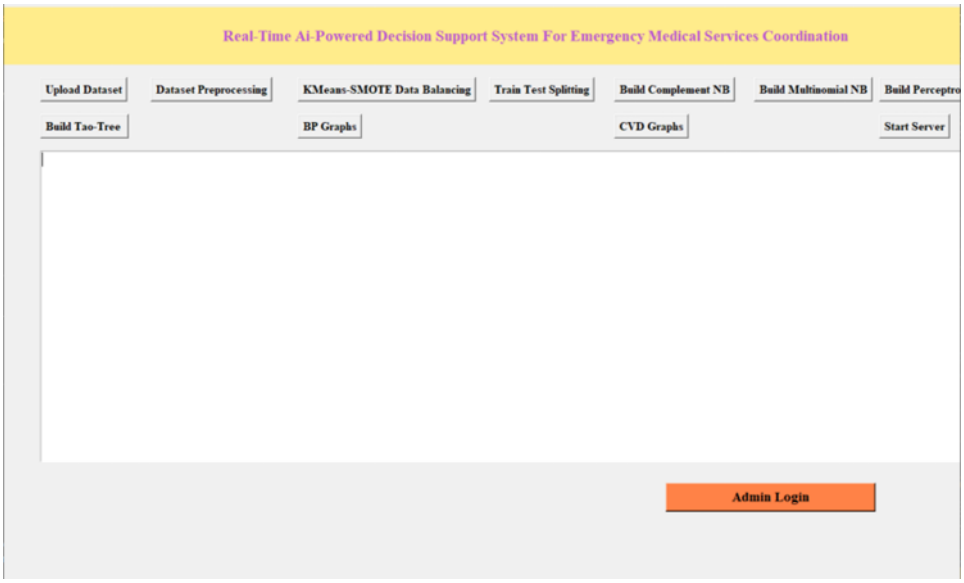


Figure 5.1 : Illustration of proposed real-time decision support system GUI application.

5.2 Upload Dataset:

Fig. 5.2 shows the dashboard after a dataset has been successfully uploaded. The previously empty white panel now displays a detailed tabular preview of the loaded data, labeled [9034 rows × 22 columns] at the bottom. The table includes key health-related features such as Sex, Age, Weight (kg), Height (m), Diastolic BP, Blood Pressure Category, Estimated LDL (mg/dL), and CVD Risk Score, among others. Sample patient records are visible, including a 32-year-old female (Row 0) with elevated blood pressure (79.0 diastolic), LDL of 140.0 mg/dL, and CVD risk score of 17.930; a 55-year-old female (Row 1) with Hypertension Stage 1; and a male patient (Row 2) with missing age and weight values (NaN) but normal blood pressure. Further down, rows such as 9029 to 9033 show varied profiles with some missing data and hypertension categories ranging from Elevated to Hypertension Stage 2.

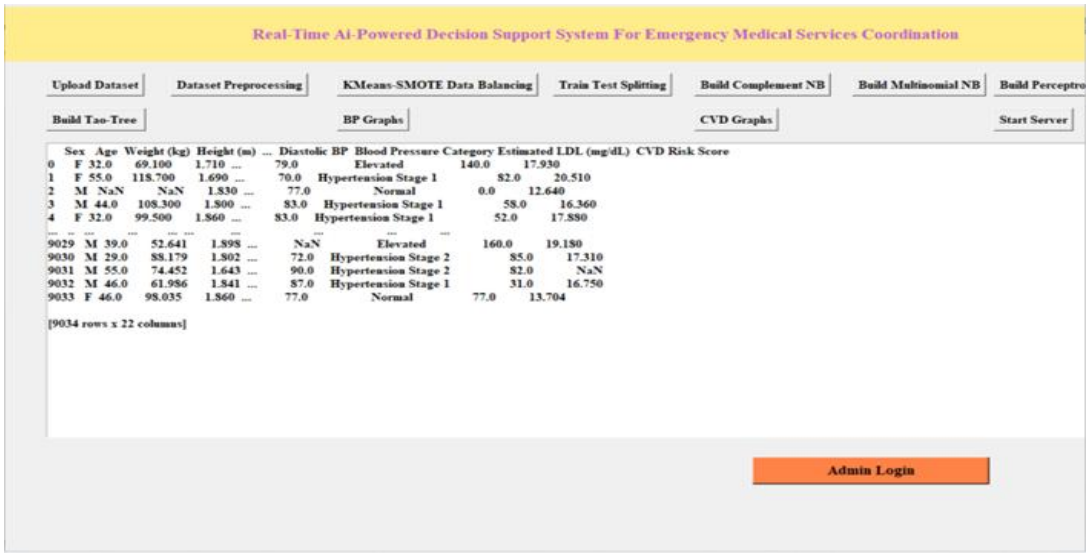
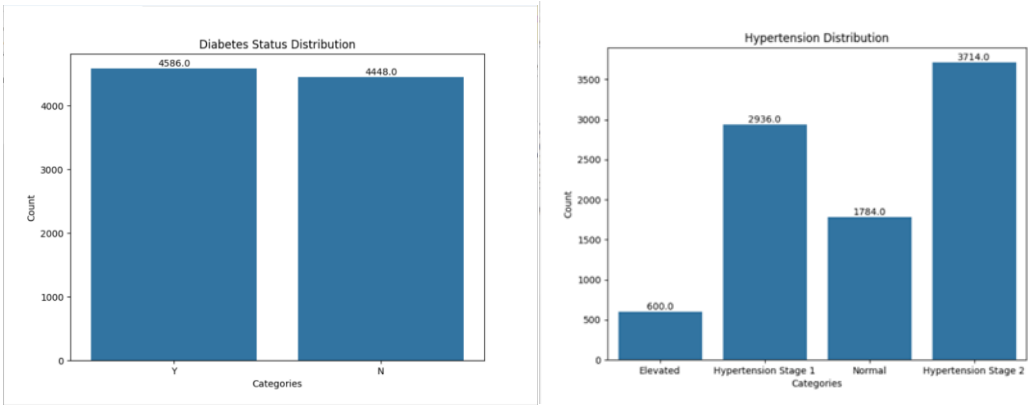


Figure 5.2 : Upload Dataset

5.3 Shows the "Diabetes Status Distribution" and "Hypertension Distribution":

Fig. 5.3 (a) shows the "Diabetes Status Distribution", presents the distribution of diabetes presence in the dataset using two categories. The Y (Yes) category has a count of 4,586.0, while the N (No) category has 4,448.0. Both bars are solid blue and nearly equal in height, indicating a slightly imbalanced but close-to-even split between patients with and without diabetes. The y-axis represents Count ranging from 0 to 4,000+, and the x-axis is labeled Categories with Y and N.

Fig. 5.3 (b) shows the "Hypertension Distribution" displays the frequency of four hypertension categories. From left to right: Elevated with 600.0 cases, Hypertension Stage 1 with 2,936.0, Normal with 1,784.0, and Hypertension Stage 2 with 3,714.0. The bars are blue and vary significantly in height, clearly illustrating a highly imbalanced distribution. Hypertension Stage 2 is the most prevalent, followed by Stage 1, then Normal, with Elevated being the least common. The y-axis shows Count from 0 to 3,500+, and the x-axis lists the four Categories.



(a) (b)

Figure 5.3 : Count plot of (a) diabetes status. (b) hypertension.

5.4 Shows the Tao-Tree classifier for binary diabetes prediction:

Fig. 5.4 shows the Tao-Tree classifier for binary diabetes prediction. Rows represent True Class (bottom to top: Y, N), and columns represent Predicted Class (left to right: Y, N). The diagonal shows correct predictions: True Y → Predicted Y: 951 (bright yellow), True N → Predicted N: 867 (lime green). Off-diagonal errors are minimal: only 8 cases of Y misclassified as N, and 10 cases of N misclassified as Y. The color scale ranges from dark purple (~200) to

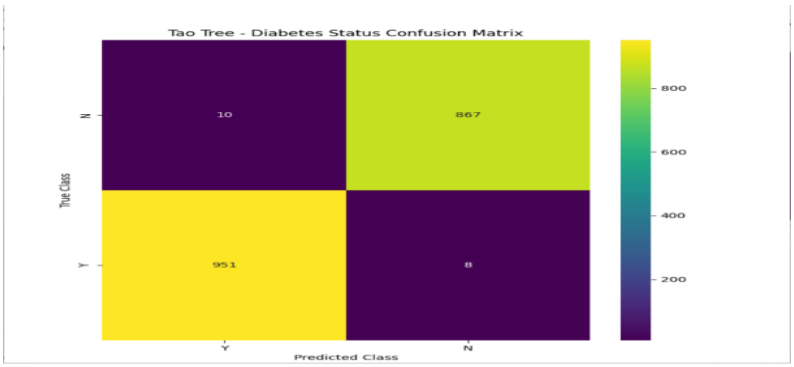


Figure 5.4 : Confusion matrices obtained with hypertension data using Proposed Tao-tree model

bright yellow (~800), highlighting the model's outstanding accuracy with just 18 total misclassifications.

5.5 Presents the Real-Time AI-Powered Decision Support System For Emergency Medical Services Coordination dashboard:

Fig. 5.5 presents the Real-Time AI-Powered Decision Support System For Emergency Medical Services Coordination dashboard after the server has been successfully started. The yellow header displays the system title as before. The horizontal navigation toolbar remains unchanged with all workflow buttons: Upload Dataset, Dataset Preprocessing, KMeans-SMOTE Data Balancing, Train Test Splitting, Build Complement NB, Build Multinomial NB, Build Perceptron, Build Tao-Tree, BP Graphs, CVD Graphs, and Start Server. The main content area, previously blank or used for data previews and graphs, now shows a single line of text: "Server Started at http://0.0.0.0:5000", indicating that the backend prediction server is active and listening on the local network at port 5000. The orange Admin Login button remains at the bottom right, suggesting that administrative access is still required for further control or monitoring. This state confirms that the system is now operational and ready for real-time inference or API-based decision support in emergency medical coordination.

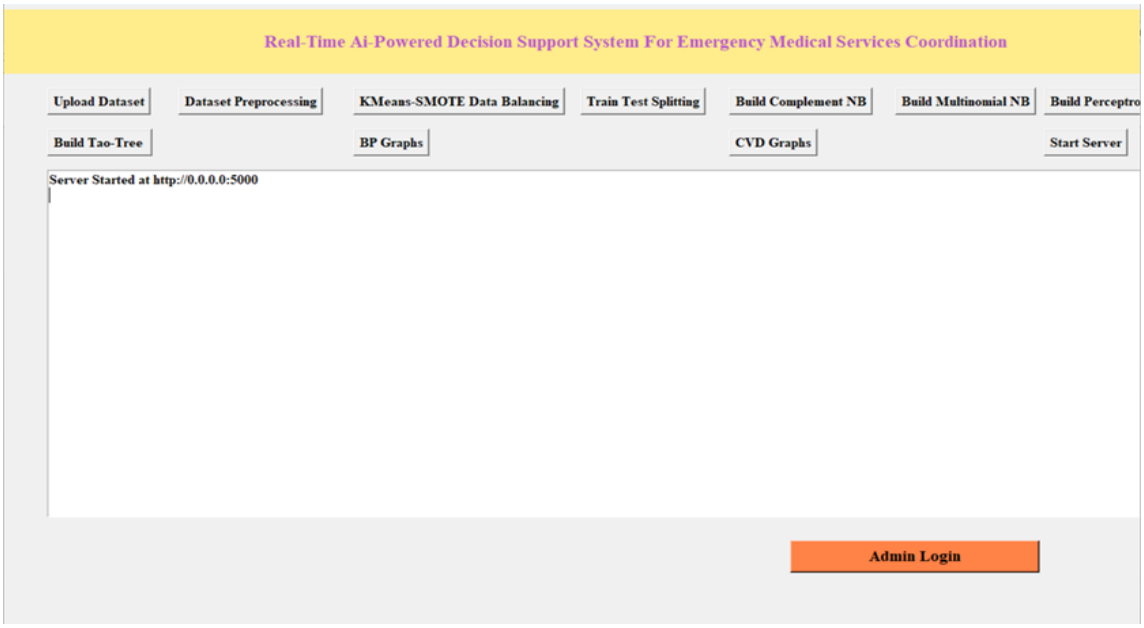


Figure 5.5 : Start server for decision support system admin panel.

6.VALIDATION

6. VALIDATION

The validation of this project primarily relies on extensive testing and well-defined test cases to ensure the accuracy and effectiveness of the inappropriate content detection system. The testing process involves multiple stages, including dataset validation, model performance evaluation, and real-world testing. By implementing a structured validation approach, we can ensure that the system consistently delivers high accuracy in detecting inappropriate content while minimizing false positives and false negatives.

6.1 INTRODUCTION

The validation phase is an important step in evaluating the effectiveness and reliability of the proposed Real-Time AI-Powered Decision Support System for Emergency Medical Services (EMS) Coordination. The purpose of validation is to ensure that the developed system produces accurate predictions and performs efficiently in real-time healthcare scenarios.

In this system, validation is carried out by testing the trained machine learning models using separate test datasets that were not used during the training phase. The dataset containing medical parameters related to hypertension and diabetes is divided into training and testing sets. The trained models, including Complement Naive Bayes, Multinomial Naive Bayes, Perceptron, and the proposed Tao Tree Classifier, are evaluated to determine their predictive performance.

To measure the effectiveness of the system, several performance metrics are used, such as Accuracy, Precision, Recall, and F1-Score. These metrics help in understanding how well the system correctly identifies patients with hypertension or diabetes and how effectively it avoids incorrect predictions. The results from these evaluations are compared across different algorithms to identify the most reliable model.

The Tao Tree Classifier demonstrates improved performance in terms of prediction accuracy and classification efficiency. Therefore, it is selected as the final model for deployment in the real-time system. The validated model is then integrated into the user interface, allowing healthcare professionals to input patient data and receive immediate predictions regarding hypertension and diabetes status. Through this validation process, the system ensures high reliability, improved prediction accuracy, and effective real-time decision support, which can assist emergency medical services in making faster and more informed healthcare decisions.

6.2 TEST CASES

TABLE 6.2.1 UPLOADING DATASET

Test case ID	Test case name	Purpose	Test Case	Output
1	User uploads Dataset.	To verify that the system successfully uploads the EMS dataset.	User uploads patient and emergency dataset into the system	Dataset successfully loaded.

TABLE 6.2.2 CLASSIFICATION

Test case ID	Test case name	Purpose	Input	Output
1	User Authentication Test	To verify that the system allows valid users to log in successfully.	Valid username and password entered in login page	User successfully logs into the system and dashboard is displayed.
2	Data Balancing Test	To verify the functionality of KMeans SMOTE data balancing	Imbalanced medical dataset	Balanced dataset generated with equal class distribution

7.CONCLUSION & FUTURE ASPECTS

7. CONCLUSION & FUTURE ASPECTS

In conclusion, the project has successfully achieved its objectives, showcasing significant progress and outcomes. The implementation and execution phases were meticulously planned and executed, leading to substantial improvements and insights. Looking ahead, the future aspects of the project hold immense potential. Future developments will focus on expanding the scope, integrating new technologies, and enhancing sustainability. These advancements will not only strengthen the existing framework but also open new avenues for growth and innovation, ensuring the project remains relevant and impactful in the long term. This strategic approach will drive continuous improvement and success.

7.1 PROJECT CONCLUSION

The study successfully demonstrates the effectiveness of an intelligent, data-driven approach for supporting medical decision-making in emergency scenarios. By integrating advanced preprocessing techniques, class imbalance handling, and multiple classification models, the system is able to accurately analyse patient health data and generate reliable predictions for critical conditions such as blood pressure category and diabetes status. The comparative evaluation highlights the strengths and limitations of different algorithms, providing a clear understanding of their suitability for healthcare applications. The inclusion of a tree-based learning approach further enhances prediction stability and interpretability, making the outcomes more transparent for practical use. The system's ability to deliver real-time predictions through an interactive interface and web-based service emphasizes its applicability in time-sensitive medical environments. Overall, the work confirms that the adoption of artificial intelligence can significantly improve diagnostic support, reduce response time, and contribute to more informed and effective healthcare decision-making.

7.2 FUTURE ASPECTS

The system has strong potential for further enhancements to make it even more practical and impactful in real-world emergency medical environments. Future improvements can include integrating deep learning models for higher prediction accuracy, expanding the system to handle multiple additional health parameters, and connecting it with IoT-based patient monitoring devices for continuous real-time data collection. A cloud-based deployment can

enable large-scale availability and faster model inference, while integrating Electronic Health Records (EHR) can improve personalized predictions. Additionally, adding advanced visualization dashboards, mobile app interfaces, and automated alert mechanisms can make this framework a complete AI-powered health monitoring and emergency response system.

8.BIBLIOGRAPHY

8. BIBLIOGRAPHY

8.1 REFERENCES

[1] O'Cathain, A., Webber, E., Nicholl, J., Munro, J., & Knowles, E. NHS Direct: consistency of triage outcomes. Emergency Medicine Journal, 20, 289-292.

[2] Kuttan, N., Pundkar, A., Gadkari, C., Patel, A., & Kumar, A. (2025). Transforming emergency medicine with artificial intelligence: From triage to clinical decision support. Multidisciplinary Reviews.

[3] Masoud, M.P., Nejad, M.K., Darebaghi, H., Chavoshi, M., & Farahani, M. (2018). The Decision Support System and Conventional Method of Telephone Triage by Nurses in Emergency Medical Services: A Comparative Investigation. Int. J. E Bus. Res., 14, 77-88.

[4] Bashiri, A., Alizadeh Savareh, B., & Ghazisaeedi, M. (2019). Promotion of prehospital emergency care through clinical decision support systems: opportunities and challenges. Clinical and Experimental Emergency Medicine, 6, 288-296.

[5] Kanani, S., Mahdavi, I., Ziaie, N., & Rahimpour Cami, B. (2024). Optimization of referral system for providing medical services to cardiac patients with cardiogenic shock manifestation under uncertainty. Scientia Iranica.

[6] Almadani, B.; Kaisar, H.; Thoker, I.R.; Aliyu, F. A Systematic Survey of Distributed Decision Support Systems in Healthcare. Systems 2025, 13, 157. https://doi.org/10.3390/systems13030157

[7] Aityan, S.K.; Mosaddegh, A.; Herrero, R.; Inchingolo, F.; Nguyen, K.C.D.; Balzanelli, M.; Lazzaro, R.; Iacovazzo, N.; Cefalo, A.; Carriero, L.; et al. Integrated AI Medical Emergency Diagnostics Advising System. Electronics 2024, 13, 4389. https://doi.org/10.3390/electronics13224389

[8] Neira-Rodado, D.; Paz-Roa, J.C.; Escobar, J.W. Emergency Medical Services Strategic Design: A Comprehensive Multiobjective Approach to Ensure System Sustainability and Quality. Smart Cities 2025, 8, 52. https://doi.org/10.3390/smartcities8020052

[9] Cvetkovic, V.M.; Tanasić, J.; Renner, R.; Roković, V.; Beriša, H. Comprehensive Risk Analysis of Emergency Medical Response Systems in Serbian Healthcare: Assessing Systemic Vulnerabilities in Disaster Preparedness and Response. Healthcare 2024, 12, 1962. https://doi.org/10.3390/healthcare12191962

[10] Strauss, C.; Schmid, M.; Kliem, D.; Müller, M. Insights from a Decade of Optimizing Emergency Medical Services Across Three Major Regions in Switzerland. Emerg. Care Med. 2024, 1, 368-381. https://doi.org/10.3390/ecm1040036

[11] Liu, J.; Dong, C.; An, S.; Mai, Q. Dynamic Evolution Analysis of the Emergency Collaboration Network for Compound Disasters: A Case Study Involving a Public Health Emergency and an Accident Disaster during COVID-19. Healthcare 2022, 10, 500. https://doi.org/10.3390/healthcare10030500

[12] Robakowska, M.; Ślęzak, D.; Żuratyński, P.; Tyrańska-Fobke, A.; Robakowski, P.; Prędkiewicz, P.; Zorena, K. Possibilities of Using UAVs in Pre-Hospital Security for Medical Emergencies. Int. J. Environ. Res. Public Health 2022, 19, 10754. https://doi.org/10.3390/ijerph191710754

[13] kuttan, N., Pundkar, A., Gadkari, C., Patel, A., & Kumar, A. (2025). Transforming emergency medicine with artificial intelligence: From triage to clinical decision support . Multidisciplinary Reviews, 8(10), 2025285. https://doi.org/10.31893/multirev.2025285

[14] Al-Araji, Z. J., Hasan, B. T., Mutleg, A. A., Benameur, N., & Cengiz, K. (2025). Improving Early Stroke Diagnosis in Emergency Medicine Using AI-Powered Clinical Decision Support. Journal of Artificial Intelligence in Medical Applications, 1(1).

[15] Kiran Kumar Jaghni. (2025). HealthNavAI: An AI-Powered National Platform for Real-Time Healthcare Service Availability and Patient Routing. Journal of Computer Science and Technology Studies, 7(10), 426-443. https://doi.org/10.32996/jcsts.2025.7.10.43

[16] Schmidt Batista, A. (2025). The Transforming Clinical Practice: The Role of AI-Powered Medical Assistants in Enhancing Healthcare Efficiency and Decision-Making. Journal of Next-Generation Research 5.0, 1(4). https://doi.org/10.70792/jngr5.0.v1i4.121

[17] Lujak, M., Billhardt, H. & Ossowski, S. Distributed coordination of emergency medical service for angioplasty patients. Ann Math Artif Intell 78, 73–100 (2016). https://doi.org/10.1007/s10472-016-9507-9

[18] Tang, X.; Zou, A.; Zhang, Z.; Zhao, Y.; Zhang, X.; Cohan, A.; Gerstein, M.B. MedAgents: Large Language Models as Collaborators for Zero-shot Medical Reasoning. *arXiv* 2023, arXiv:2311.10537.

[19] Miller, M.; Jorm, L.; Partyka, C.; Burns, B.; Habig, K.; Oh, C.; Immens, S.; Ballard, N.; Gallego, B. Identifying prehospital trauma patients from ambulance patient care records; comparing two methods using linked data in New South Wales, Australia. *Injury* 2024, *55*, 111570

[20] Elfahim, O.; Edjinedja, K.L.; Cossus, J.; Youssfi, M.; Barakat, O.; Desmettre, T. A Systematic Literature Review of Artificial Intelligence in Prehospital Emergency Care. *Big Data Cogn. Comput.* 2025, *9*, 219. https://doi.org/10.3390/bdcc9090219

[21] Zota, R. D., Cîmpeanu, I. A., & Lungu, M. A. (2025). Exploring AI in Healthcare Systems: A Study of Medical Applications and a Proposal for a Smart Clinical Assistant. *Electronics*, *14*(18), 3727. https://doi.org/10.3390/electronics14183727

[22] Nickel, S.; Reuter-Oppermann, M.; Saldanha-Da-Gama, F. Ambulance location under stochastic demand: A sampling approach. *Oper. Res. Health Care* 2016, *8*, 24–32.

[23] Neira-Rodado, D.; Paz-Roa, J.C.; Escobar, J.W. Emergency Medical Services Strategic Design: A Comprehensive Multiobjective Approach to Ensure System Sustainability and Quality. *Smart Cities* 2025, *8*, 52. https://doi.org/10.3390/smartcities8020052

8.2 GITHUB LINK

https://github.com/GOJJI-USHASRI/Real-time-EMS-coordination